Projector-Generated Special Functions I

Polyphase Differential Lifts, Bernoulli-Lerch Wavelets, Cyclotomic Detail Functions, and Möbius Coherence

Komorebi P.

Abstract

This paper develops a special-function theory generated by exact projector and polyphase operations. For an analytic scalar function, residue sections are assembled into a vector-valued packet function. Multiplication and the Euler derivative become a carry-shift matrix and a diagonal differential operator that satisfy the same Weyl commutator as the scalar pair. Every ordered scalar differential equation therefore lifts to an exact matrix differential system.

The construction is applied to exponentials, Bessel functions, Airy functions, generalized hypergeometric equations, Bernoulli interpolation, Hurwitz zeta, Lerch transcendent, and Möbius-weighted reciprocal L-function transfers. A complex-order Bernoulli-Hurwitz packet family is proved to obey exact reconstruction, exact parent-child refinement, an order-lowering differential ladder, a translation law, and integer interpolation of the terms in Raabe multiplication. A unitary transform across the child address yields cyclotomic Bernoulli-Lerch details with an inverse transform and Parseval identity. The two-channel case identifies Bernoulli and Euler polynomials as the low-pass and high-pass outputs of one exact refinement bank.

Packet Wronskians provide a direct test for ratio monotonicity, zero interlacing, and Hermite-Biehler compatibility. Computation also supplies a useful negative result: the first Bernoulli-Euler Wronskian family is not universally real-rooted. The final part introduces a generalized Möbius-Bernoulli transfer function. Its binomial, Dirichlet, exponential-generating, Mellin, wavelet, polyphase, character, and quantum forms are proved. The diagonal packet energy has a closed asymptotic law. In the zeta case, the Riemann hypothesis is equivalent to subpolynomial growth of a normalized packet-coherence ratio. This equivalence is a theorem target, not a proof of the hypothesis.

All figures were generated from the accompanying Python program. The release contains 26 vector figures, matching PNG files, exact tables, 51 algebraic or high-precision checks, and the full source listing.

Status and claim boundary

The paper proves the algebraic lift, the Bernoulli-Hurwitz packet laws, the cyclotomic transform, the Bernoulli-Euler channel identity, the packet-Wronskian formulas, the generalized transfer identities, the Mellin formula, the long-time power identity, and the packet-energy asymptotic under the stated hypotheses.

Classical inputs include the Hurwitz zeta function, the Lerch transcendent, Raabe multiplication, Bernoulli and Euler polynomial identities, inverse Euler products, the Riesz and Báez-Duarte criteria, and standard squarefree-density estimates.

The paper does not claim that every generated function is historically new. The contribution is the common reconstruction, refinement, differential, transform, and certificate structure. The Bernoulli interpolation and its distinction from the integer Bernoulli-value sequence were developed in the preceding transfer-block note [8]. Historical novelty of individual specializations requires a separate literature study. The packet-coherence condition in the zeta case is equivalent to RH and remains open.

Contents

Status and claim boundary **1**

I The general polyphase lift **5**

1 From a scalar germ to a vector special function **5**

2 The carry-shift representation of the Weyl algebra **7**

3 Worked lifts of classical functions **9**

3.1 The exponential 9

3.2 Bessel functions 11

3.3 Airy functions 12

3.4 Generalized hypergeometric functions 12

II Bernoulli-Hurwitz packets and cyclotomic details **13**

4 Complex-order Bernoulli interpolation **13**

5 Exact refinement, differentiation, and translation **14**

6 Cyclotomic Bernoulli-Lerch details **16**

7 Exponential generating function of the detail bank **18**

8 Bernoulli and Euler as low-pass and high-pass outputs **18**

9 Packet Wronskians and zero diagnostics **19**

10 A worked cyclotomic norm over the third roots of unity **21**

III Möbius-Bernoulli transfer functions 23

11 A generalized reciprocal L-function transfer family 23

12 Exponential generating and Mellin transforms 23

13 The zeta specialization and the Bernoulli seed 24

14 Scale localization and the Mertens wavelet 25

15 Diagonal packet energy 27

16 Möbius packet coherence and RH 29

17 Long-time mean power and a quantum correlation 31

18 Arithmetic polyphase and character packets 33

IV Worked computational atlas and research program 35

19 Worked example: complete three-channel exponential lift 35

20 Worked example: Bernoulli packet refinement 36

21 Worked example: third-root cyclotomic norm 36

22 Worked example: the principal transfer at order 1200 37

23 Algorithmic workflow 37

24 Open problems generated by the calculus 38

24.1 Near-term problems 38

24.2 Major-conjecture interfaces 38

25 Verification 39

26 Conclusion 40

A Formula registry 40

B Complete Python source 41

C Bibliographic note 57

List of Figures

1.1 A coefficient sequence separated into four residue packets. Each Taylor exponent appears in exactly one row, so the analysis map loses no coefficient. 6

2.1 The carry-shift action for five packets. Multiplication advances the packet address, while the last packet wraps around with the factor u. 8

2.2 The operator-level lift. The scalar differential equation and the packet matrix system are two coordinate realizations of the same ordered algebra. 9

3.1 Exponential packets for two and three channels. The section functions are ordinary generalized hypergeometric functions arranged as one exact vector object. 10

3.2 Numerical reconstruction residuals for three packet counts. The residual remains at floating-point roundoff across the displayed interval. 11

3.3 Three residue sections of J_0. The scalar oscillation is divided by exponent class, while exact synthesis recovers the original Bessel function. 12

4.1 Four Bernoulli-Hurwitz packets and their synthesis. The packets may be individually large or oscillatory, but their complete sum equals the direct interpolation. 14

5.1 A Bernoulli packet refined from modulus one to modulus eight. Every parent is the exact sum of its children. The refinement law works at complex order, not only at integer samples. 15

6.1 Four cyclotomic Bernoulli-Lerch channels as the translation parameter varies. The non-DC details carry phase and magnitude information that disappears in the parent sum. 17

6.2 Energy before and after the cyclotomic transform. The child and detail energies agree to numerical precision, so the transform is a genuine perfect-reconstruction bank. 17

8.1 Two Bernoulli children transformed into Bernoulli and Euler channels. The transform changes coordinates without changing the information carried by the pair. 19

9.1 Four low-order packet Wronskians. The zeros can be read directly from the ratio derivative and provide a concrete interlacing diagnostic. 20

9.2 Real and nonreal roots for the first fifteen packet Wronskians. The finite computation rejects the naive conjecture that this entire Wronskian family is real-rooted. 21

10.1 Growth of the exact $L = 3$ cyclotomic detail norm. The parity-dependent square pattern follows from Bernoulli reflection symmetry. 22

14.1 Rescaled principal transfer kernels. The curves collapse near one common profile after the coordinate is divided by $\sqrt{k}$. 25

14.2 The Mertens wavelet at three scales. The sign change remains fixed in the normalized coordinate, making the scale selection explicit. 26

15.1 The scaled energy $k^{3/2}E_k$. The numerical curve approaches the exact constant predicted by the theorem. 28

15.2 Active squarefree mass and effective dimension. The number of packets contributing at one scale grows like a constant multiple of $\sqrt{k}$. 29

16.1 The principal transfer amplitude, diagonal energy scale, and coherence ratio. The numerical values are finite diagnostics only. They do not establish the required asymptotic bound. . 30

16.2 The normalized sequence $k^{3/4}c_k$ in logarithmic time. Zeros off the critical line would appear as expanding modes in this coordinate. 31

17.1 One finite Möbius-Bernoulli correlation and its running mean power. The dashed line is the exact diagonal energy of the truncated packet family. 32

17.2 Operator realization of the generalized transfer function. Scale evolution and real-time logarithmic evolution commute because both are diagonal in the integer basis. 33

18.1 Residue packets of the principal Möbius transfer at three scales. Positive and negative packets shift with k, while their exact sum remains c_k. 34

18.2 Principal and quadratic-character transfer amplitudes. The character coordinate can strongly alter cancellation even when the same scale filter is used. 35

23.1 The generated function families and their main transfer routes. The same packet operations produce differential systems, refinable Bernoulli packets, cyclotomic details, Wronskians, and arithmetic coherence signals. 38

24.1 Status map for the paper. The map separates proved identities, classical inputs, finite computations, open special-function questions, and major conjecture interfaces. 39

Part I

The general polyphase lift

Chapter 1

From a scalar germ to a vector special function

Let

$$F(z) = \sum_{n=0}^{\infty} a_n z^n$$

be analytic in a neighborhood of the origin. Fix an integer $M \geq 2$.

Definition 1.1 (Residue-section functions)**.** For $0 \leq r < M$, define

$$\boxed{F_{M,r}(u) = \sum_{m=0}^{\infty} a_{Mm+r} u^m.}$$

The packet vector is

$$\mathbf{F}_M(u) = \begin{pmatrix} F_{M,0}(u) \\ F_{M,1}(u) \\ \vdots \\ F_{M,M-1}(u) \end{pmatrix}.$$

Theorem 1.2 (Exact polyphase reconstruction)**.** *Wherever the defining series converge,*

$$\boxed{F(z) = \sum_{r=0}^{M-1} z^r F_{M,r}(z^M).}$$

Proof. Every exponent has a unique representation $n = Mm + r$ with $0 \le r < M$. Therefore

$$\sum_{n\ge 0} a_n z^n = \sum_{r=0}^{M-1} \sum_{m\ge 0} a_{Mm+r} z^{Mm+r} = \sum_{r=0}^{M-1} z^r F_{M,r}(z^M).$$

□

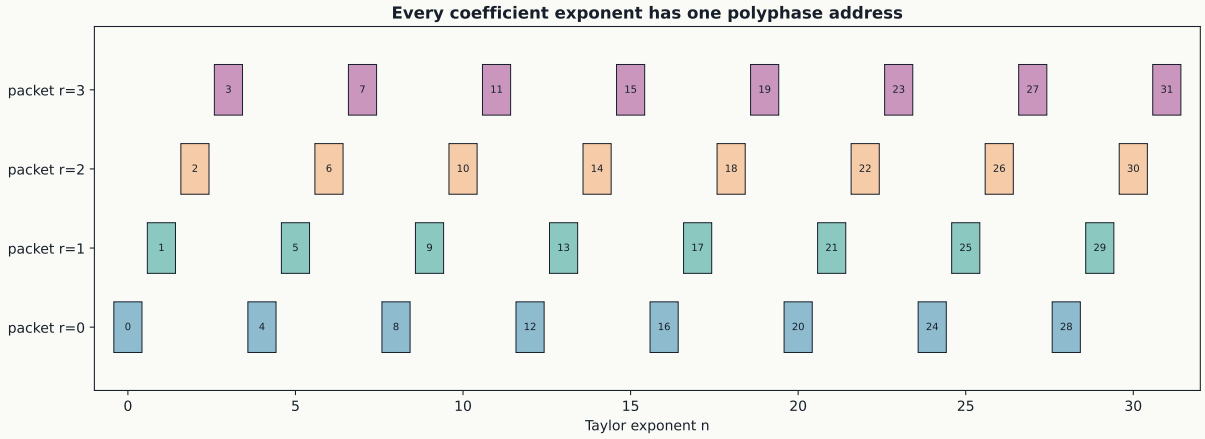


Figure 1.1: A coefficient sequence separated into four residue packets. Each Taylor exponent appears in exactly one row, so the analysis map loses no coefficient.

This elementary identity becomes more useful when the scalar differential operators are also transported into packet coordinates.

Chapter 2

The carry-shift representation of the Weyl algebra

Define the carry-shift matrix

$$C_M(u) = \begin{pmatrix} 0 & 0 & \cdots & 0 & u \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}.$$

Define the packet Euler operator

$$\Theta_M = Mu\frac{\mathrm{d}}{\mathrm{d}u}I_M + \mathrm{diag}(0, 1, \ldots, M-1).$$

Proposition 2.1 (Intertwining laws)**.** *Let*

$$\mathcal{S}_M\mathbf{f}(z) = \sum_{r=0}^{M-1} z^r f_r(z^M).$$

Then

$$\boxed{\mathcal{S}_M(C_M\mathbf{f}) = z\,\mathcal{S}_M\mathbf{f}}$$

and

$$\boxed{\mathcal{S}_M(\Theta_M\mathbf{f}) = \left(z\frac{\mathrm{d}}{\mathrm{d}z}\right)\mathcal{S}_M\mathbf{f}.}$$

Proof. Multiplication by z sends packet r to packet $r+1$. The last packet wraps to packet 0 and contributes a factor $z^M = u$. This is exactly $C_M(u)$. For the differential identity, a monomial u^m in packet r reconstructs to z^{Mm+r}. The operator $Mu\frac{\mathrm{d}}{\mathrm{d}u} + r$ multiplies that monomial by $Mm+r$, which is the eigenvalue of $z\frac{\mathrm{d}}{\mathrm{d}z}$ on the reconstructed monomial. □

Corollary 2.2 (Weyl commutator)**.**

$$\boxed{[\Theta_M, C_M] = C_M.}$$

Proof. The scalar operators satisfy

$$\left[z\frac{\mathrm{d}}{\mathrm{d}z}, z\right] = z.$$

Apply the two intertwining laws. The synthesis map is injective because the residue decomposition is unique. □

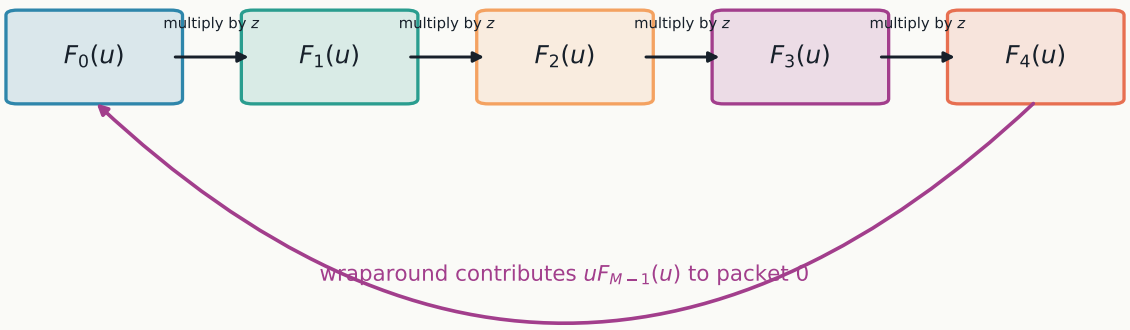


Figure 2.1: The carry-shift action for five packets. Multiplication advances the packet address, while the last packet wraps around with the factor u.

Theorem 2.3 (Differential lift)**.** *Let $\mathcal{L}(z,\theta)$ be an ordered polynomial in the noncommuting variables z and θ, where $\theta = z\frac{\mathrm{d}}{\mathrm{d}z}$. If*

$$\mathcal{L}(z,\theta)F = 0,$$

then

$$\boxed{\mathcal{L}(C_M,\Theta_M)\mathbf{F}_M = 0.}$$

The order of the factors in every monomial of $\mathcal{L}$ is preserved.

Proof. The assignment

$$z \longmapsto C_M, \qquad \theta \longmapsto \Theta_M$$

preserves the defining commutator. It therefore gives a representation of the same ordered operator algebra. Intertwining each ordered monomial with $\mathcal{S}_M$ gives

$$\mathcal{S}_M\big(\mathcal{L}(C_M,\Theta_M)\mathbf{F}_M\big) = \mathcal{L}(z,\theta)F = 0.$$

Injectivity of $\mathcal{S}_M$ gives the result. □

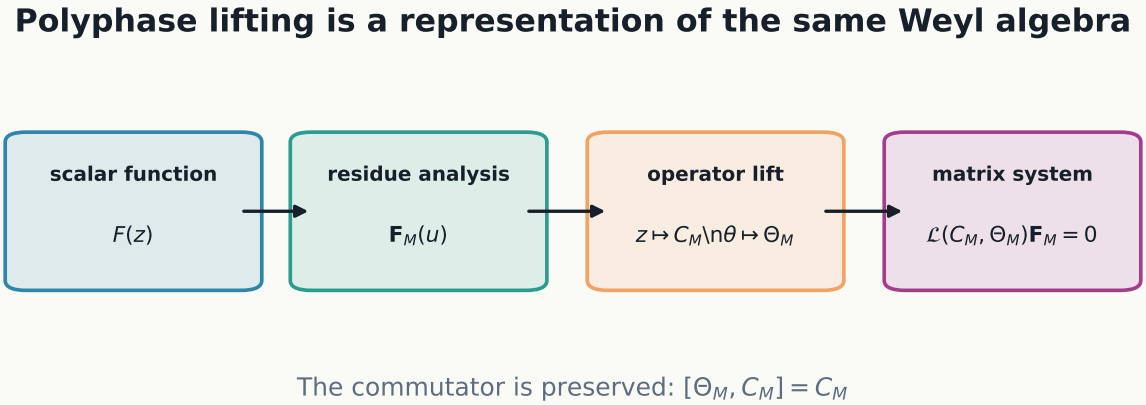


Figure 2.2: The operator-level lift. The scalar differential equation and the packet matrix system are two coordinate realizations of the same ordered algebra.

Chapter 3

Worked lifts of classical functions

3.1 The exponential

The scalar equation

$$\theta e^z = z e^z$$

lifts to

$$\boxed{\Theta_M \mathbf{E}_M = C_M \mathbf{E}_M.}$$

The residue sections are

$$E_{M,r}(u) = \sum_{m=0}^{\infty} \frac{u^m}{(Mm+r)!}.$$

Gauss multiplication gives

$$\boxed{E_{M,r}(u) = \frac{1}{r!}{}_0F_{M-1}\left(\left.\begin{matrix}-\\ \frac{r+1}{M}, \ldots, \widehat{1}, \ldots, \frac{r+M}{M}\end{matrix}\right| \frac{u}{M^M}\right),}$$

where the denominator parameter equal to 1 is omitted.

Worked example 3.1 (Two channels)**.** For $M = 2$,

$$E_{2,0}(u) = \cosh\sqrt{u}, \qquad E_{2,1}(u) = \frac{\sinh\sqrt{u}}{\sqrt{u}}.$$

Hence

$$e^z = E_{2,0}(z^2) + zE_{2,1}(z^2) = \cosh z + \sinh z.$$

Worked example 3.2 (Three channels)**.** For $M = 3$,

$$E_{3,0}(u) = {}_0F_2\left(;\frac{1}{3},\frac{2}{3};\frac{u}{27}\right),$$
$$E_{3,1}(u) = {}_0F_2\left(;\frac{2}{3},\frac{4}{3};\frac{u}{27}\right),$$
$$E_{3,2}(u) = \frac{1}{2}{}_0F_2\left(;\frac{4}{3},\frac{5}{3};\frac{u}{27}\right).$$

The exact synthesis is

$$\boxed{e^z = E_{3,0}(z^3) + zE_{3,1}(z^3) + z^2E_{3,2}(z^3).}$$

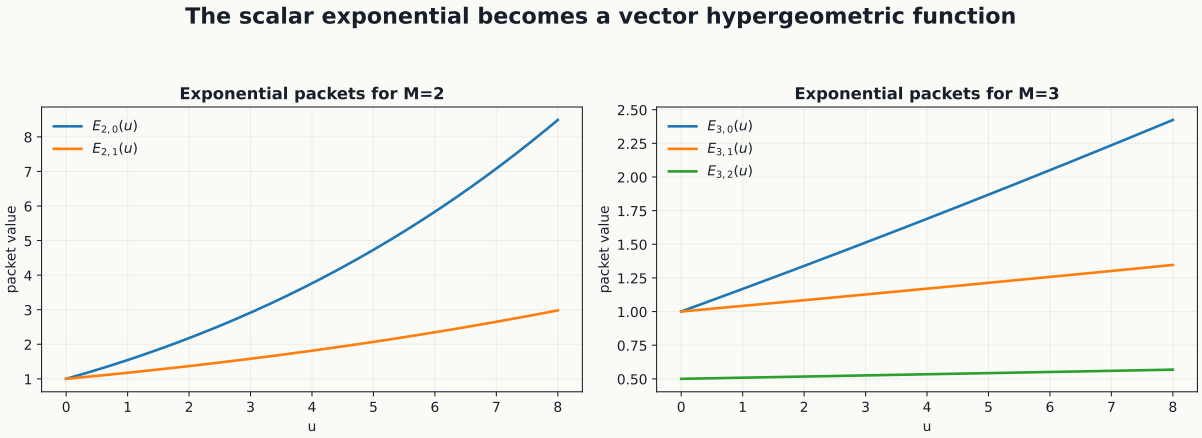


Figure 3.1: Exponential packets for two and three channels. The section functions are ordinary generalized hypergeometric functions arranged as one exact vector object.

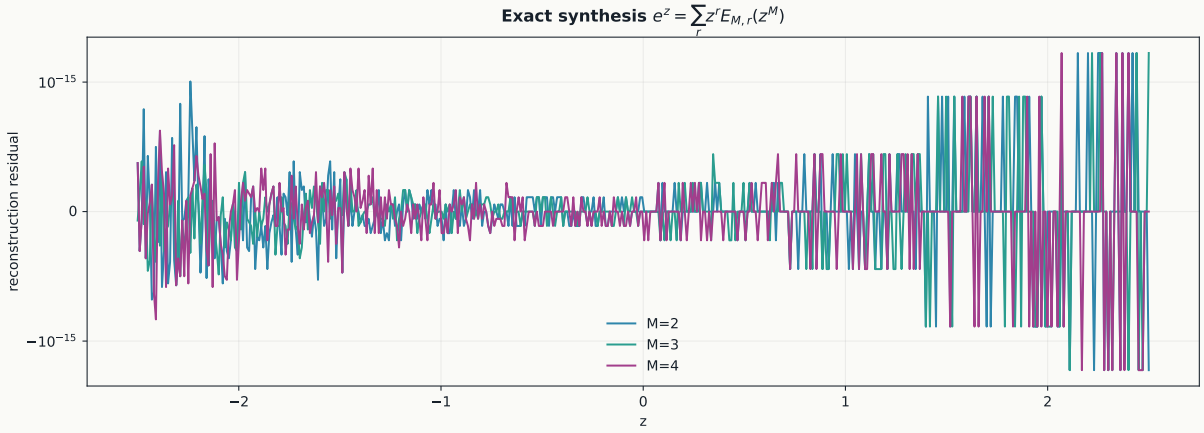


Figure 3.2: Numerical reconstruction residuals for three packet counts. The residual remains at floating-point roundoff across the displayed interval.

3.2 Bessel functions

The Bessel equation for J_ν can be written

$$\left(\theta^2 + z^2 - \nu^2\right) J_\nu(z) = 0.$$

Whenever the chosen branch is analytic in the working neighborhood, the packet vector satisfies

$$\boxed{\left(\Theta_M^2 + C_M^2 - \nu^2 I\right) \mathbf{J}_{\nu,M} = 0.}$$

For integer order, the Taylor expansion at the origin supplies a global entire packet vector. Figure 3.3 shows the three residue sections of J_0.

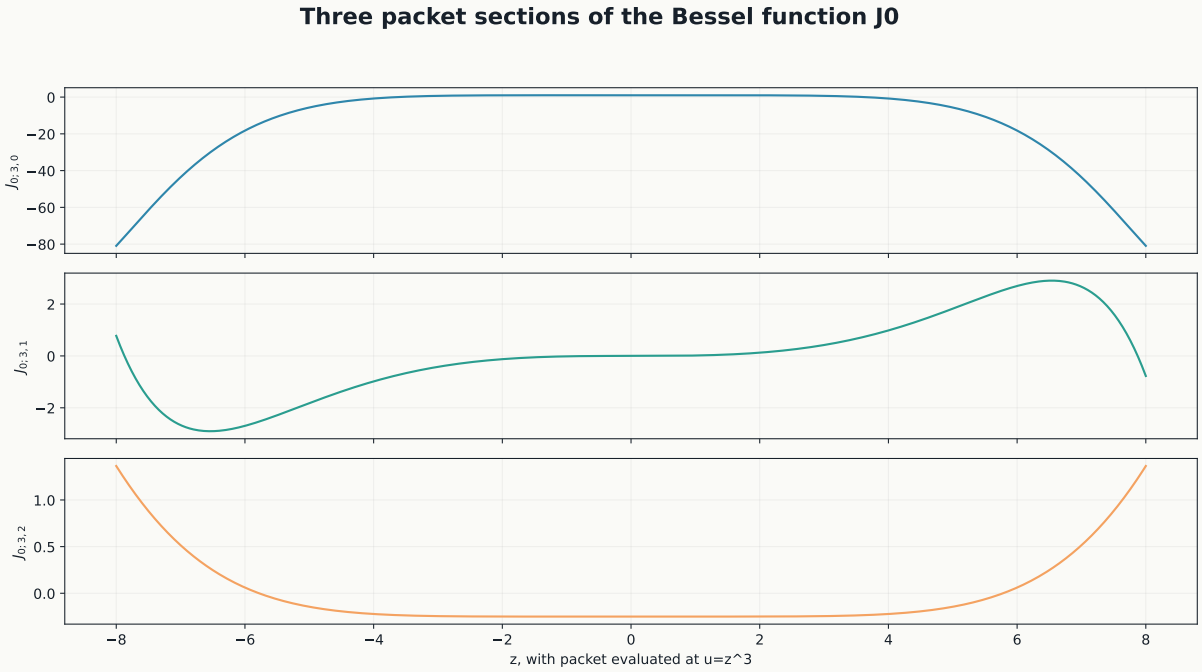


Figure 3.3: Three residue sections of J_0. The scalar oscillation is divided by exponent class, while exact synthesis recovers the original Bessel function.

3.3 Airy functions

The Airy equation $F'' = zF$ is equivalent to

$$\theta(\theta - 1)F = z^3 F.$$

Its packet lift is

$$\boxed{\Theta_M(\Theta_M - I)\mathbf{F}_M = C_M^3 \mathbf{F}_M.}$$

This example is useful because the scalar equation contains a derivative order and a multiplication order that do not commute. The ordered lift preserves both.

3.4 Generalized hypergeometric functions

The generalized hypergeometric function satisfies

$$\left[\theta \prod_{j=1}^{q}(\theta + b_j - 1) - z\prod_{i=1}^{p}(\theta + a_i)\right] F = 0.$$

The packet system is therefore

$$\boxed{\left[\Theta_M \prod_{j=1}^{q}(\Theta_M + (b_j - 1)I) - C_M \prod_{i=1}^{p}(\Theta_M + a_i I)\right] \mathbf{F}_M = 0.}$$

This gives a systematic matrix-valued lift of every scalar generalized hypergeometric equation.

Part II

Bernoulli-Hurwitz packets and cyclotomic details

Chapter 4

Complex-order Bernoulli interpolation

For $a > 0$, define

$$\boxed{\mathfrak{B}(z;a) = -z\zeta(1-z,a).}$$

The pole of the Hurwitz zeta function at $1 - z = 1$ is removed by the factor $-z$, so $\mathfrak{B}(z;a)$ is entire in z and $\mathfrak{B}(0;a) = 1$. The special-value and multiplication formulas used below are classical properties of Bernoulli polynomials and the Hurwitz zeta function [1, 2].

For positive integers n,

$$\mathfrak{B}(n;a) = B_n(a).$$

Thus $\mathfrak{B}$ is a complex-order interpolation of the Bernoulli polynomial sequence.

Definition 4.1 (Bernoulli-Hurwitz packet)**.** For $M \geq 1$ and $0 \leq r < M$, define

$$\boxed{\mathfrak{B}_{M,r}(z;a) = -zM^{z-1}\zeta\left(1-z, \frac{a+r}{M}\right).}$$

At $z = 0$, define $\mathfrak{B}_{M,r}(0;a) = 1/M$ by removal of the singularity.

Theorem 4.2 (Reconstruction)**.**

$$\boxed{\mathfrak{B}(z;a) = \sum_{r=0}^{M-1} \mathfrak{B}_{M,r}(z;a).}$$

Proof. The Hurwitz multiplication formula is

$$\sum_{r=0}^{M-1} \zeta\left(s, \frac{a+r}{M}\right) = M^s\zeta(s,a).$$

Set $s = 1 - z$ and multiply by $-zM^{z-1}$. □

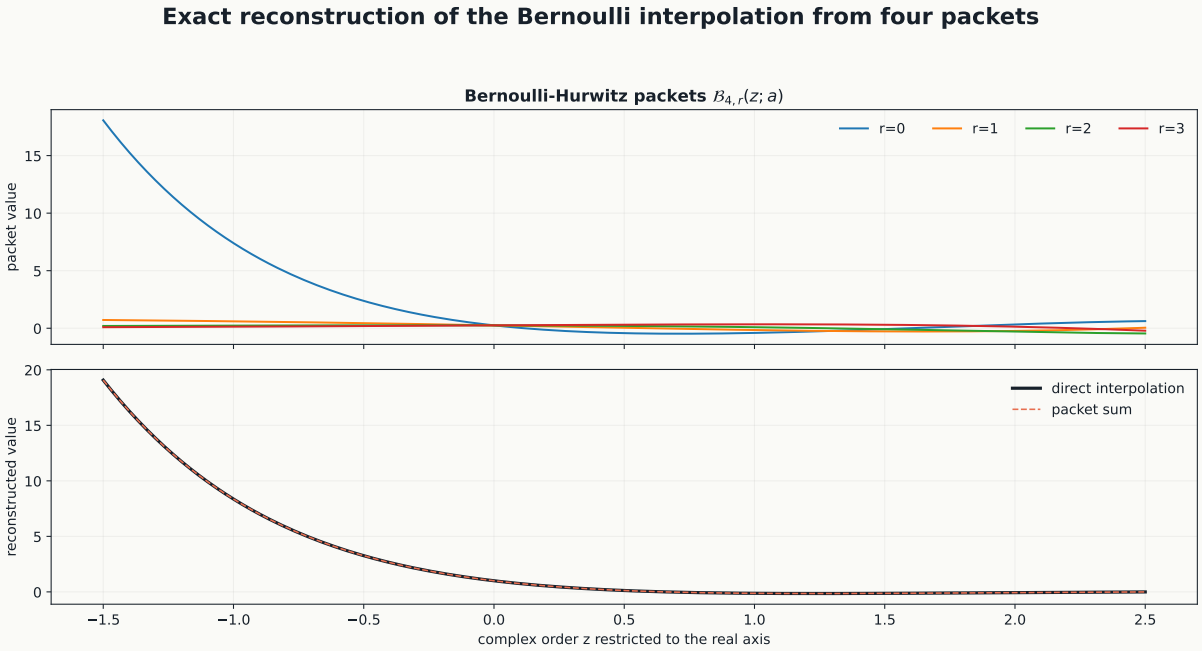


Figure 4.1: Four Bernoulli-Hurwitz packets and their synthesis. The packets may be individually large or oscillatory, but their complete sum equals the direct interpolation.

Chapter 5

Exact refinement, differentiation, and translation

Theorem 5.1 (Parent-child refinement)**.** *For integers $L, M \geq 1$,*

$$\boxed{\mathfrak{B}_{M,r}(z;a) = \sum_{\ell=0}^{L-1} \mathfrak{B}_{LM,r+\ell M}(z;a).}$$

Proof. Set $A = (a+r)/M$. The child arguments are $(A+\ell)/L$. Summing their Hurwitz zeta functions and applying the multiplication formula at modulus L gives the parent. The powers of L cancel exactly. □

Theorem 5.2 (Integer samples)**.** *For every positive integer n,*

$$\boxed{\mathfrak{B}_{M,r}(n;a) = M^{n-1} B_n\left(\frac{a+r}{M}\right).}$$

Proof. Use $\zeta(1-n,x) = -B_n(x)/n$. □

The reconstruction theorem at integer order becomes Raabe multiplication:

$$B_n(a) = M^{n-1} \sum_{r=0}^{M-1} B_n\left(\frac{a+r}{M}\right).$$

Theorem 5.3 (Differential ladder)**.**

$$\boxed{\frac{\partial}{\partial a}\mathfrak{B}_{M,r}(z;a) = z\mathfrak{B}_{M,r}(z-1;a).}$$

Proof. The Hurwitz derivative is

$$\frac{\partial}{\partial x}\zeta(s,x) = -s\zeta(s+1,x).$$

Apply the chain rule to $x = (a+r)/M$ and simplify. □

Theorem 5.4 (Translation law)**.**

$$\boxed{\mathfrak{B}_{M,r}(z;a+M) - \mathfrak{B}_{M,r}(z;a) = z(a+r)^{z-1}.}$$

Proof. Use

$$\zeta(s,x+1) = \zeta(s,x) - x^{-s}.$$

□

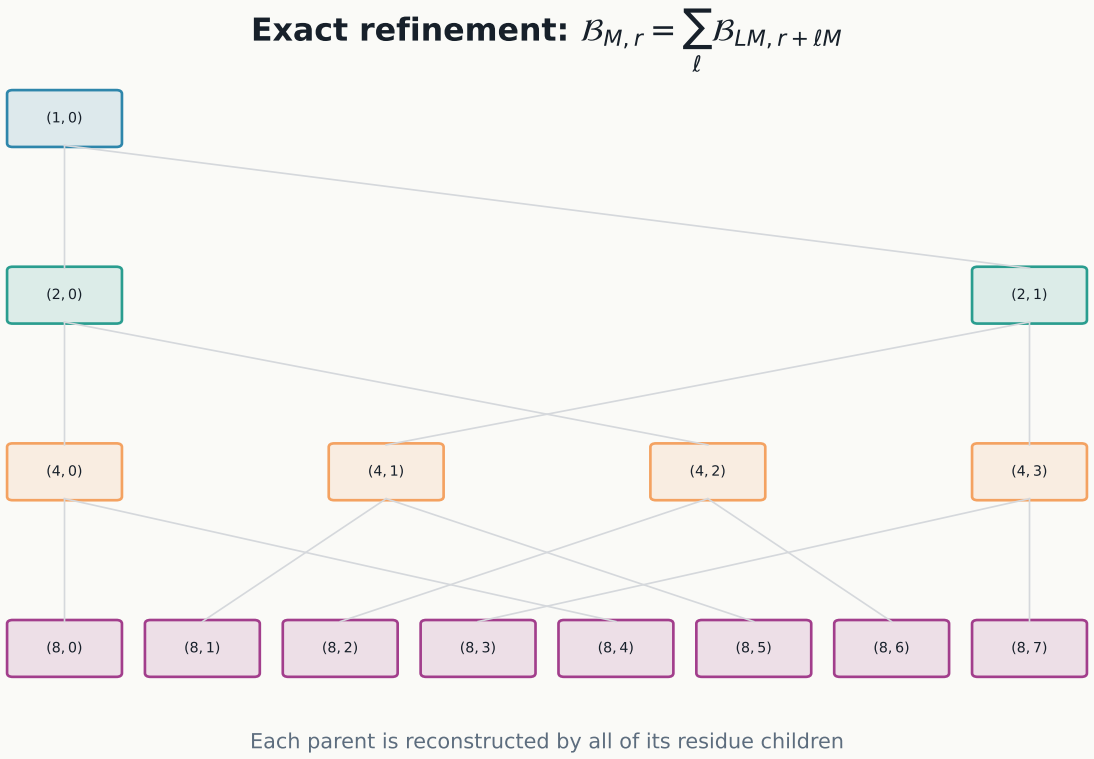


Figure 5.1: A Bernoulli packet refined from modulus one to modulus eight. Every parent is the exact sum of its children. The refinement law works at complex order, not only at integer samples.

Chapter 6

Cyclotomic Bernoulli-Lerch details

Let $\omega_L = \mathrm{e}^{2\pi \mathrm{i}/L}$.

Definition 6.1 (Cyclotomic detail)**.**

$$\boxed{\mathcal{W}_{M,r}^{(L,j)}(z;a) = \frac{1}{\sqrt{L}} \sum_{\ell=0}^{L-1} \omega_L^{-j\ell} \mathfrak{B}_{LM,r+\ell M}(z;a).}$$

Theorem 6.2 (Lerch closed form)**.**

$$\boxed{\mathcal{W}_{M,r}^{(L,j)}(z;a) = -\frac{z}{\sqrt{L}} M^{z-1} \Phi\left(\omega_L^{-j}, 1-z, \frac{a+r}{M}\right).}$$

Proof. Write each child Hurwitz zeta function as a series. The child address ℓ and the internal summation index combine into one integer $n = Lm + \ell$. The Fourier weight $\omega_L^{-j\ell}$ becomes ω_L^{-jn}. The resulting series is the Lerch transcendent. □

The DC channel is the parent:

$$\boxed{\mathcal{W}_{M,r}^{(L,0)} = \frac{1}{\sqrt{L}} \mathfrak{B}_{M,r}.}$$

The inverse transform is

$$\boxed{\mathfrak{B}_{LM,r+\ell M} = \frac{1}{\sqrt{L}} \sum_{j=0}^{L-1} \omega_L^{j\ell} \mathcal{W}_{M,r}^{(L,j)}.}$$

For each fixed (z, a), Parseval gives

$$\boxed{\sum_{\ell=0}^{L-1} \left|\mathfrak{B}_{LM,r+\ell M}\right|^2 = \sum_{j=0}^{L-1} \left|\mathcal{W}_{M,r}^{(L,j)}\right|^2.}$$

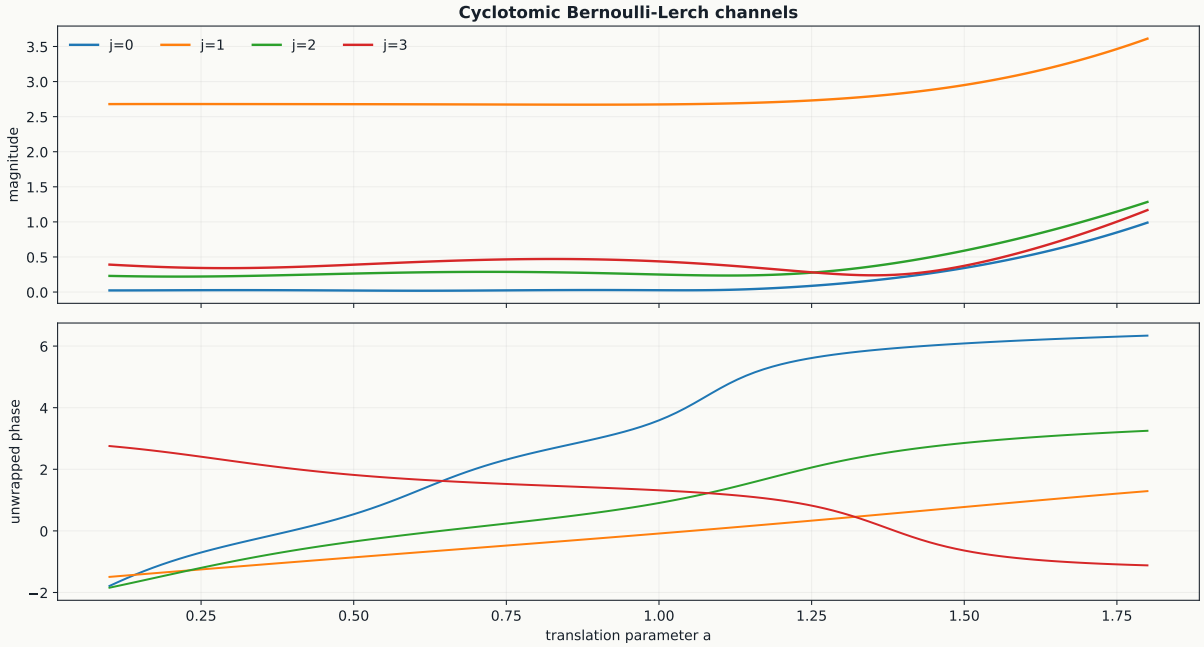


Figure 6.1: Four cyclotomic Bernoulli-Lerch channels as the translation parameter varies. The non-DC details carry phase and magnitude information that disappears in the parent sum.

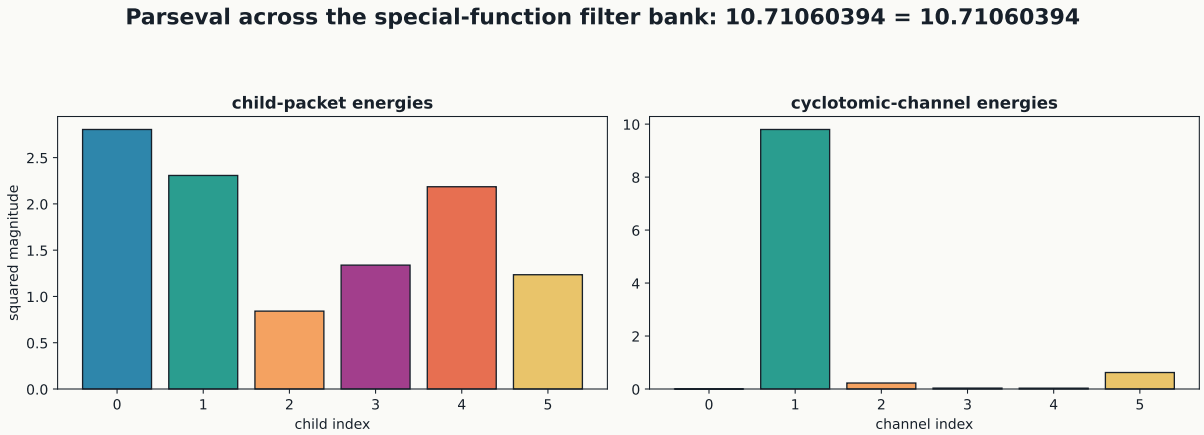


Figure 6.2: Energy before and after the cyclotomic transform. The child and detail energies agree to numerical precision, so the transform is a genuine perfect-reconstruction bank.

Chapter 7

Exponential generating function of the detail bank

At positive integer order, define

$$W_{L,j}(n;a) = \frac{1}{\sqrt{L}} \sum_{\ell=0}^{L-1} \omega_L^{-j\ell} L^{n-1} B_n\left(\frac{a+\ell}{L}\right).$$

Theorem 7.1 (Cyclotomic EGF)**.**

$$\boxed{\sum_{n=0}^{\infty} W_{L,j}(n;a)\frac{t^n}{n!} = \frac{te^{at}}{\sqrt{L}\left(\omega_L^{-j}e^t - 1\right)}.}$$

Proof. The EGF of the ℓ-th child is

$$\sum_{n\geq 0} L^{n-1} B_n\left(\frac{a+\ell}{L}\right)\frac{t^n}{n!} = \frac{te^{(a+\ell)t}}{e^{Lt}-1}.$$

Take the finite Fourier transform over ℓ and sum the geometric progression. The factor $1 - e^{Lt}$ cancels the denominator. □

This is the standard root-of-unity Apostol-Bernoulli generating form, now obtained as one channel of an exact refinement bank. The Lerch representation uses the conventional analytic continuation described in [2].

Chapter 8

Bernoulli and Euler as low-pass and high-pass outputs

Take $L = 2$. The two children are

$$2^{n-1}B_n\left(\frac{a}{2}\right) \quad \text{and} \quad 2^{n-1}B_n\left(\frac{a+1}{2}\right).$$

Theorem 8.1 (Two-channel identity)**.**

$$\boxed{W_{2,0}(n;a) = \frac{B_n(a)}{\sqrt{2}}}$$

and, for $n \geq 1$*,*

$$\boxed{W_{2,1}(n;a) = -\frac{n}{2\sqrt{2}}E_{n-1}(a).}$$

Proof. The first formula is Raabe multiplication. The second follows from

$$E_{n-1}(a) = \frac{2^n}{n}\left[B_n\left(\frac{a+1}{2}\right) - B_n\left(\frac{a}{2}\right)\right].$$

□

Thus Bernoulli and Euler polynomials are the DC and Nyquist channels of one exact Bernoulli refinement.

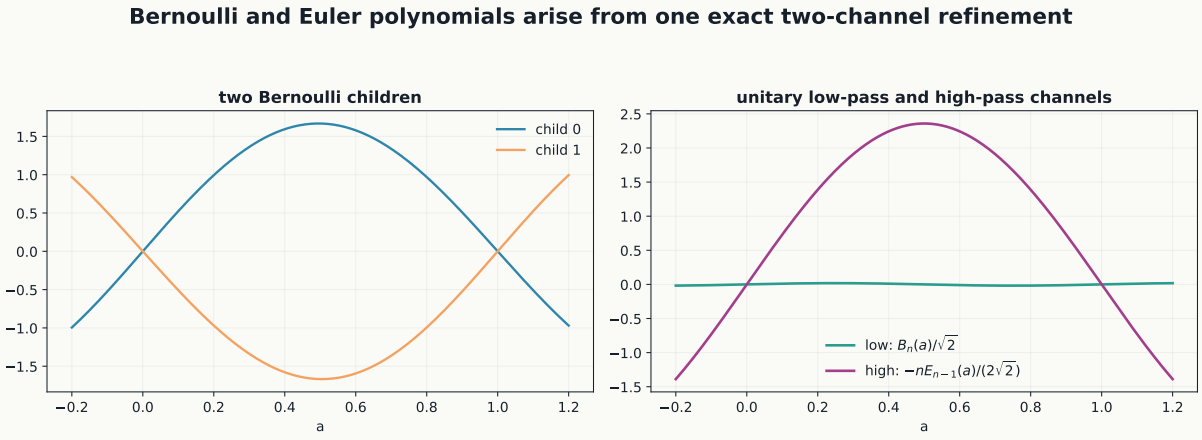


Figure 8.1: Two Bernoulli children transformed into Bernoulli and Euler channels. The transform changes coordinates without changing the information carried by the pair.

Chapter 9

Packet Wronskians and zero diagnostics

Define

$$\boxed{\mathcal{D}_{j,\ell}(z;a) = \mathcal{W}_j(z;a)\mathcal{W}_\ell(z-1;a) - \mathcal{W}_\ell(z;a)\mathcal{W}_j(z-1;a).}$$

Proposition 9.1 (Ratio derivative)**.** *Where* $\mathcal{W}_\ell(z;a) \neq 0$,

$$\boxed{\frac{\partial}{\partial a}\left(\frac{\mathcal{W}_j(z;a)}{\mathcal{W}_\ell(z;a)}\right) = -\frac{z\mathcal{D}_{j,\ell}(z;a)}{\mathcal{W}_\ell(z;a)^2}.}$$

Proof. Differentiate the ratio and use the order-lowering differential ladder for both channels. □

For the two-channel integer family, define

$$\boxed{\mathcal{W}_n(a) = nB_{n-1}(a)E_{n-1}(a) - (n-1)B_n(a)E_{n-2}(a).}$$

This is four times the packet Wronskian of the Bernoulli and Euler channels at consecutive orders.

Worked example 9.2 (Orders two through four)**.**

$$\mathcal{W}_2(a) = \frac{3a^2 - 3a + 1}{3},$$

which has no real zeros. Also,

$$\mathcal{W}_3(a) = a^2(a-1)^2,$$

while

$$\mathcal{W}_4(a) = \frac{a(a-1)}{5}\left(5a^4 - 10a^3 + 5a - 2\right).$$

The fourth-order polynomial has nonreal roots.

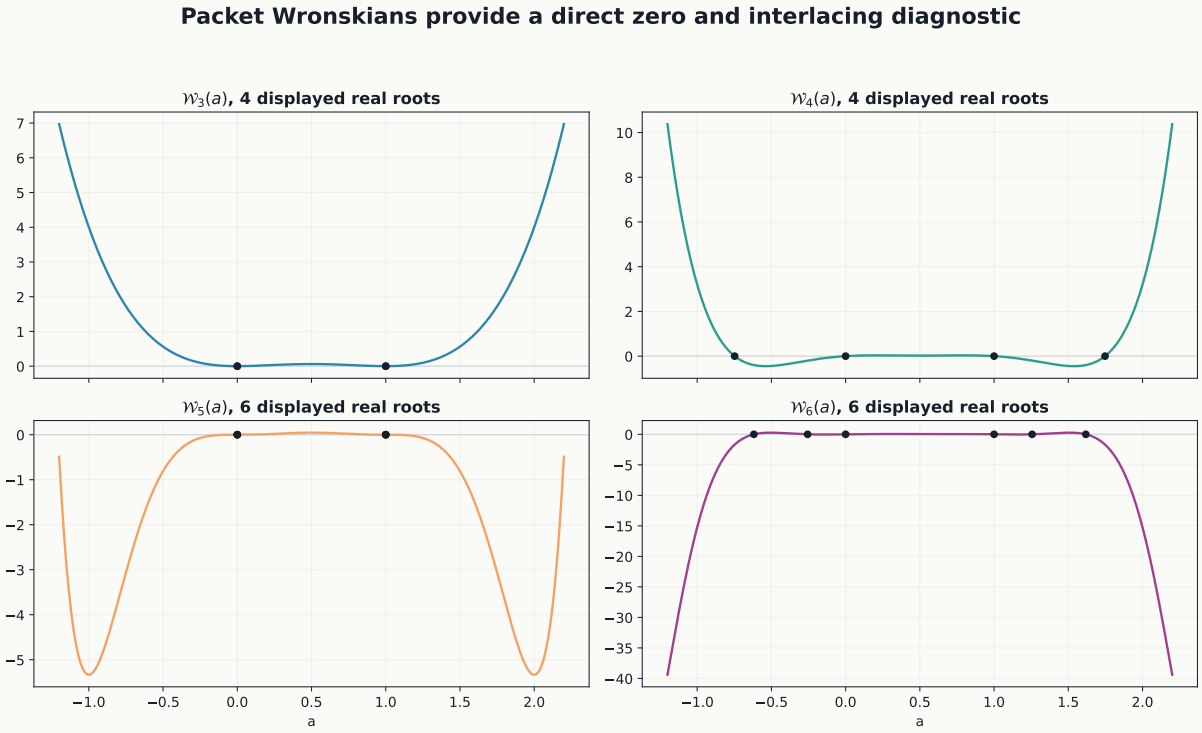


Figure 9.1: Four low-order packet Wronskians. The zeros can be read directly from the ratio derivative and provide a concrete interlacing diagnostic.

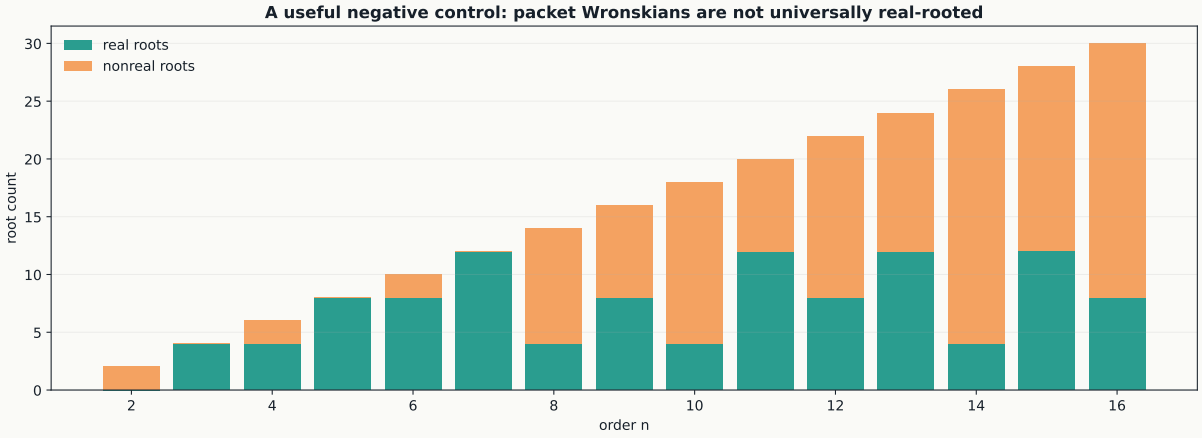


Figure 9.2: Real and nonreal roots for the first fifteen packet Wronskians. The finite computation rejects the naive conjecture that this entire Wronskian family is real-rooted.

Boundary statement 9.3 (What the negative control means). The failure of universal real-rootedness does not remove the Wronskian route. It shows that a successful theorem must impose the right channel, order, parameter, or completed-function normalization. A blanket real-rootedness statement is false.

Chapter 10

A worked cyclotomic norm over the third roots of unity

Use the unnormalized $L = 3$ detail

$$D_1(n) = \sum_{r=0}^{2} \omega_3^{-r} 3^{n-1} B_n\left(\frac{r}{3}\right).$$

Write $D_1 = A_n + B_n\omega_3$. Its field norm is

$$N_n = A_n^2 - A_n B_n + B_n^2.$$

Theorem 10.1 (Parity law for the third-root detail norm)**.** *For every even $n \geq 2$,*

$$D_1(n) = 3^{n-1}\left[B_n - B_n\left(\frac{1}{3}\right)\right] \in \mathbb{Q}$$

and

$$\boxed{N_n = \left\{3^{n-1}\left[B_n - B_n\left(\frac{1}{3}\right)\right]\right\}^2.}$$

For every odd $n \geq 3$,

$$D_1(n) = (\omega_3^2 - \omega_3)3^{n-1} B_n\left(\frac{1}{3}\right)$$

and

$$\boxed{N_n = 3\left[3^{n-1} B_n\left(\frac{1}{3}\right)\right]^2.}$$

Proof. Bernoulli reflection gives $B_n(1-x) = (-1)^n B_n(x)$. For even n, the two child values at $1/3$ and $2/3$ agree, and $\omega_3 + \omega_3^2 = -1$ reduces the detail to the displayed rational number. For odd $n \geq 3$, one has $B_n = 0$ and $B_n(2/3) = -B_n(1/3)$. Since $(\omega_3^2 - \omega_3)(\omega_3 - \omega_3^2) = 3$, the field norm is three times the indicated rational square. □

Table 10.1: Exact first non-DC cyclotomic detail data for $L = 3$ and $a = 0$.

n	A_n	B_n	$N_n = A_n^2 - A_n B_n + B_n^2$	factor pattern
1	$-1/3$	$1/3$	$1/3$	first-order exception
2	$2/3$	0	$4/9$	$9N_n$ is a square
3	$-1/3$	$-2/3$	$1/3$	$3N_n$ is a square
4	$-4/3$	0	$16/9$	$9N_n$ is a square
5	$5/3$	$10/3$	$25/3$	$3N_n$ is a square
6	$26/3$	0	$676/9$	$9N_n$ is a square
7	$-49/3$	$-98/3$	$2401/3$	$3N_n$ is a square
8	$-328/3$	0	$107584/9$	$9N_n$ is a square
9	$809/3$	$1618/3$	$654481/3$	$3N_n$ is a square
10	$6710/3$	0	$45024100/9$	$9N_n$ is a square
11	$-20317/3$	$-40634/3$	$412780489/3$	$3N_n$ is a square
12	$-201772/3$	0	$40711939984/9$	$9N_n$ is a square

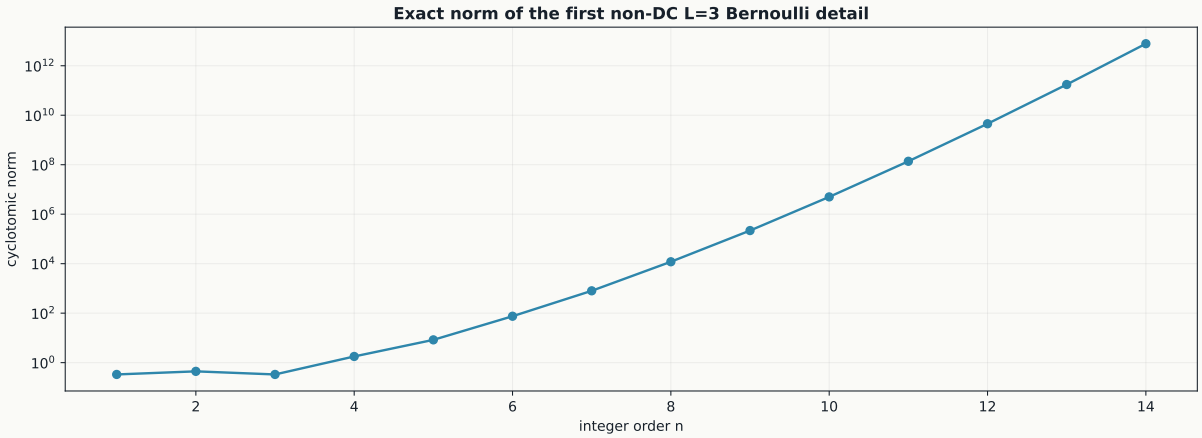


Figure 10.1: Growth of the exact $L = 3$ cyclotomic detail norm. The parity-dependent square pattern follows from Bernoulli reflection symmetry.

Part III

Möbius-Bernoulli transfer functions

Chapter 11

A generalized reciprocal L-function transfer family

Let χ be a Dirichlet character. For parameters $a > 1$, $\beta > 0$, integer $k \geq 0$, and real t, define

$$\boxed{\mathcal{C}_{a,\beta,\chi}(k,t) = \sum_{j=0}^{k} (-1)^j \binom{k}{j} \frac{1}{L(a+\beta j + \mathrm{i}t, \chi)}.}$$

Theorem 11.1 (Möbius form)**.** *In the half-plane of absolute convergence,*

$$\boxed{\mathcal{C}_{a,\beta,\chi}(k,t) = \sum_{n=1}^{\infty} \frac{\mu(n)\chi(n)}{n^{a+\mathrm{i}t}} \left(1 - \frac{1}{n^\beta}\right)^k.}$$

Proof. Use

$$\frac{1}{L(s,\chi)} = \sum_{n \geq 1} \frac{\mu(n)\chi(n)}{n^s}$$

and interchange the finite and absolutely convergent sums. The binomial sum becomes $(1 - n^{-\beta})^k$. □

Proposition 11.2 (Parameter-shift recursion)**.**

$$\boxed{\mathcal{C}_{a,\beta,\chi}(k+1,t) = \mathcal{C}_{a,\beta,\chi}(k,t) - \mathcal{C}_{a+\beta,\beta,\chi}(k,t).}$$

Proof. Apply Pascal's identity to the binomial coefficient and shift the index in the second sum. □

Chapter 12

Exponential generating and Mellin transforms

Define

$$\boxed{\mathcal{R}_{a,\beta,\chi}(x,t) = x \sum_{n=1}^{\infty} \frac{\mu(n)\chi(n)}{n^{a+\mathrm{i}t}} \mathrm{e}^{-x/n^\beta}.}$$

Theorem 12.1 (Poissonization)**.**

$$\boxed{\sum_{k=0}^{\infty} \mathcal{C}_{a,\beta,\chi}(k,t) \frac{x^k}{k!} = \frac{\mathrm{e}^x}{x} \mathcal{R}_{a,\beta,\chi}(x,t).}$$

Proof. For each integer n,

$$\sum_{k \geq 0} \left(1 - \frac{1}{n^\beta}\right)^k \frac{x^k}{k!} = \exp\left[x\left(1 - \frac{1}{n^\beta}\right)\right].$$

Sum over n. □

Theorem 12.2 (Mellin transform)**.** *For absolute convergence, assume*

$$-1 < \Re s < \frac{a-1}{\beta} - 1.$$

The resulting identity extends meromorphically wherever the reciprocal L-function side is defined.

$$\boxed{\int_0^\infty \mathcal{R}_{a,\beta,\chi}(x,t)x^{s-1}\,\mathrm{d}x = \frac{\Gamma(s+1)}{L(a+\mathrm{i}t-\beta(s+1),\chi)}.}$$

Proof. Integrate term by term. The change of variable $x = n^\beta y$ gives

$$\int_0^\infty x^s \mathrm{e}^{-x/n^\beta}\,\mathrm{d}x = n^{\beta(s+1)}\Gamma(s+1).$$

The remaining Dirichlet series is the reciprocal L-function. □

Chapter 13

The zeta specialization and the Bernoulli seed

Take the principal character, $a = 2$, $\beta = 2$, and $t = 0$. Then

$$\boxed{c_k = \mathcal{C}_{2,2}(k,0) = \sum_{j=0}^{k}(-1)^j\binom{k}{j}\frac{1}{\zeta(2j+2)}.}$$

The even zeta values satisfy

$$\frac{1}{\zeta(2j+2)} = \frac{2(2j+2)!}{(2\pi)^{2j+2}\,|B_{2j+2}|}.$$

If

$$q = (2\pi)^{-2}, \qquad A_j = \frac{2(2j+2)!}{|B_{2j+2}|},$$

then

$$\boxed{c_k = q(I-qE)^k A_0.}$$

The Báez-Duarte theorem [3, 4] states that RH is equivalent to

$$\boxed{c_k = O_\varepsilon(k^{-3/4+\varepsilon}) \quad \text{for every } \varepsilon > 0.}$$

Thus RH is a stability statement for repeated weighted finite differences of one normalized Bernoulli seed.

Chapter 14

Scale localization and the Mertens wavelet

Write

$$f_k(x) = x^{-2}(1 - x^{-2})^k.$$

Then

$$c_k = \sum_{n\geq 1} \mu(n) f_k(n).$$

The maximum of f_k occurs at

$$x = \sqrt{k+1}.$$

More generally, the kernel

$$x^{-a}(1 - x^{-\beta})^k$$

is centered at

$$\boxed{x_*(k) = \left(1 + \frac{\beta k}{a}\right)^{1/\beta}.}$$

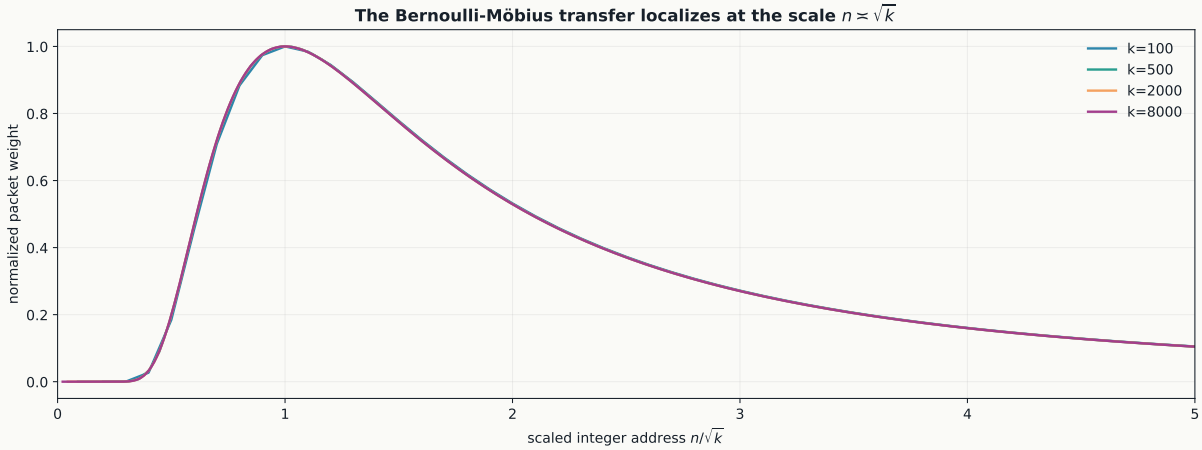


Figure 14.1: Rescaled principal transfer kernels. The curves collapse near one common profile after the coordinate is divided by $\sqrt{k}$.

Let

$$M(x) = \sum_{n\leq x} \mu(n).$$

Discrete summation by parts gives

$$\boxed{c_k = \sum_{n=1}^{\infty} M(n)\left[f_k(n) - f_k(n+1)\right].}$$

The continuous wavelet is

$$\boxed{\psi_k(x) = -f_k'(x) = 2x^{-5}(1 - x^{-2})^{k-1}(x^2 - k - 1).}$$

It has one sign change at $x = \sqrt{k+1}$ and zero integral.

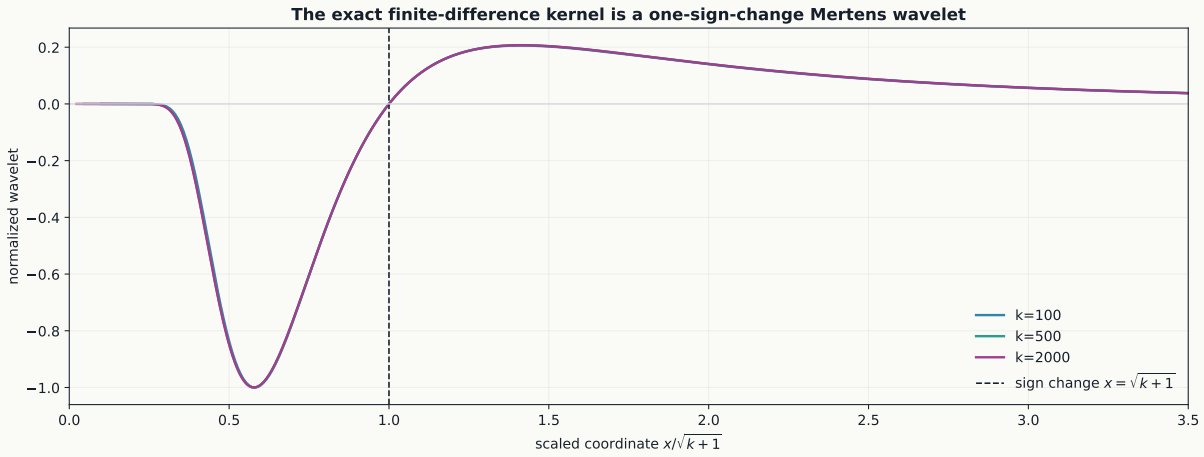


Figure 14.2: The Mertens wavelet at three scales. The sign change remains fixed in the normalized coordinate, making the scale selection explicit.

Theorem 14.1 (Mellin response of the wavelet)**.** *For $k \geq 1$ and $\Re s < 3$,*

$$\boxed{\int_1^\infty \psi_k(x) x^{s-1}\,\mathrm{d}x = \frac{s-1}{2}\mathrm{B}\left(\frac{3-s}{2}, k+1\right).}$$

Proof. First compute

$$\int_1^\infty f_k(x) x^{s-1}\,\mathrm{d}x = \frac{1}{2}\mathrm{B}\left(1-\frac{s}{2}, k+1\right)$$

by the substitution $y = x^{-2}$. Integration by parts transfers the derivative from f_k to x^{s-1}. □

Mellin inversion gives the exact Nörlund-Rice form

$$\boxed{c_k = \frac{1}{4\pi\mathrm{i}} \int_{\sigma-\mathrm{i}\infty}^{\sigma+\mathrm{i}\infty} \frac{\mathrm{B}(1-s/2, k+1)}{\zeta(s)}\,\mathrm{d}s, \qquad 1 < \sigma < 2.}$$

A simple zero ρ contributes asymptotically

$$\boxed{\frac{\Gamma(1-\rho/2)}{2\zeta'(\rho)} k^{\rho/2-1}.}$$

In logarithmic time $u = \log k$, an off-critical zero creates an exponentially growing normalized mode.

Chapter 15

Diagonal packet energy

Define

$$E_{a,\beta,\chi}(k) = \sum_{n=1}^{\infty} \frac{\mu(n)^2 \, |\chi(n)|^2}{n^{2a}} \left(1 - \frac{1}{n^{\beta}}\right)^{2k}.$$

For the asymptotic theorem below, assume that χ is primitive with conductor q, and set

$$\delta_{\chi} = \frac{1}{\zeta(2)} \prod_{p \mid q} \frac{p}{p+1}.$$

Theorem 15.1 (Packet-energy asymptotic)**.** *For $a > 1/2$,*

$$\boxed{E_{a,\beta,\chi}(k) \sim \frac{\delta_{\chi}}{\beta} \Gamma\left(\frac{2a-1}{\beta}\right) (2k)^{-(2a-1)/\beta}.}$$

Proof. The summatory density of squarefree integers coprime to q is

$$\sum_{n \le x} \mu(n)^2 \, |\chi(n)|^2 = \delta_{\chi} x + O_q(\sqrt{x}).$$

Use partial summation. The kernel is concentrated at $n \asymp k^{1/\beta}$. On this scale,

$$\left(1 - n^{-\beta}\right)^{2k} = \exp(-2kn^{-\beta})(1 + o(1)).$$

After the scaling $n = (2k)^{1/\beta} y$, the main term becomes

$$\delta_{\chi} (2k)^{-(2a-1)/\beta} \int_0^{\infty} y^{-2a} \mathrm{e}^{-y^{-\beta}} \, \mathrm{d}y.$$

The substitution $v = y^{-\beta}$ evaluates the integral as

$$\frac{1}{\beta} \Gamma\left(\frac{2a-1}{\beta}\right).$$

The square-root error in the summatory function is lower order after the same localization. □

For $a = \beta = 2$,

$$\boxed{E_k \sim \frac{3}{2^{5/2} \pi^{3/2}} k^{-3/2}.}$$

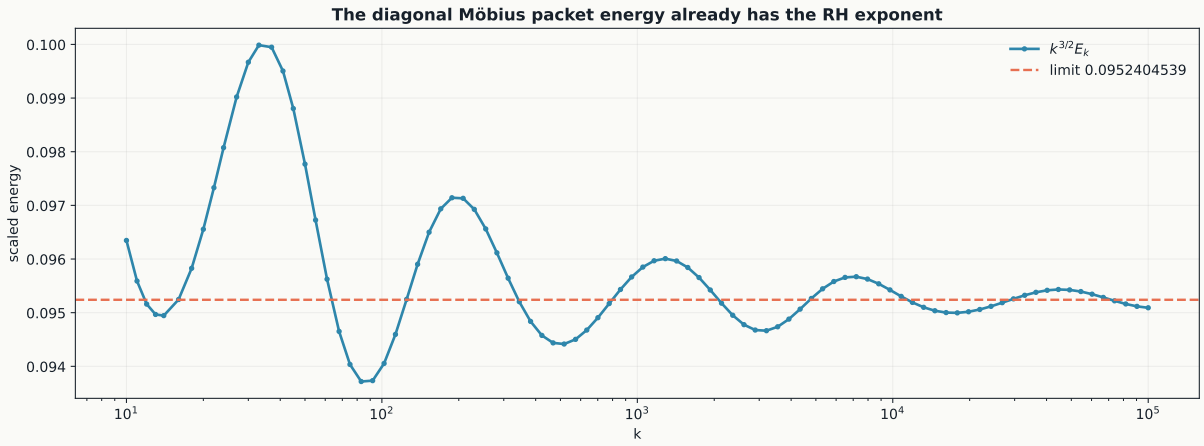


Figure 15.1: The scaled energy $k^{3/2}E_k$. The numerical curve approaches the exact constant predicted by the theorem.

Define also the active squarefree mass

$$S_k = \sum_{n\geq 1} \frac{\mu(n)^2}{n^2}\left(1-\frac{1}{n^2}\right)^k.$$

The same argument gives

$$S_k \sim \frac{3}{\pi^{3/2}}k^{-1/2}.$$

Therefore

$$\boxed{\frac{S_k^2}{E_k} \sim \frac{3\cdot 2^{5/2}}{\pi^{3/2}}\sqrt{k}.}$$

This is the effective number of active packets.

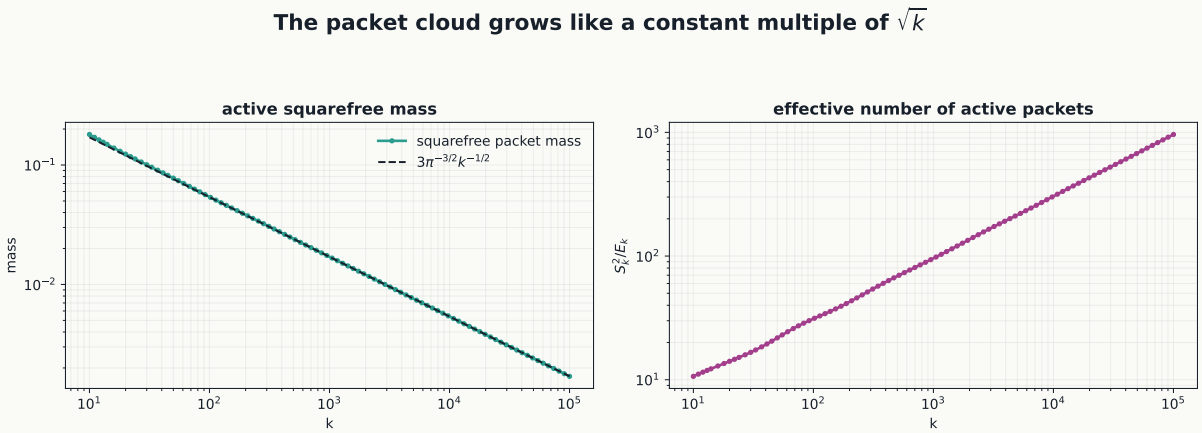


Figure 15.2: Active squarefree mass and effective dimension. The number of packets contributing at one scale grows like a constant multiple of $\sqrt{k}$.

Chapter 16

Möbius packet coherence and RH

Define

$$\boxed{\Gamma_k = \frac{|c_k|^2}{E_k}.}$$

Theorem 16.1 (Coherence form of the Báez-Duarte criterion)**.** *The Riemann hypothesis is equivalent to*

$$\boxed{\Gamma_k = O_\varepsilon(k^\varepsilon) \quad \textit{for every } \varepsilon > 0.}$$

Proof. The energy theorem gives $E_k \asymp k^{-3/2}$. If RH holds, the Báez-Duarte criterion gives $|c_k| \ll_\varepsilon k^{-3/4+\varepsilon}$. Hence $\Gamma_k \ll_\varepsilon k^{2\varepsilon}$, which is subpolynomial after renaming 2ε. Conversely, subpolynomial coherence gives

$$|c_k|^2 \ll_\varepsilon k^{-3/2+\varepsilon},$$

so $|c_k| \ll_\varepsilon k^{-3/4+\varepsilon}$ after another harmless renaming. □

The theorem separates the problem into two parts. The diagonal energy decay is already proved. The remaining RH content is the off-diagonal arithmetic coherence of the Möbius signs.

Table 16.1: Selected numerical values of the principal Möbius-Bernoulli transfer.

k	c_k	E_k	Γ_k	S_k^2/E_k
10	-6.91391e-02	3.04674e-03	1.56896e+00	1.06483e+01
50	-5.36535e-03	2.76532e-04	1.04100e-01	2.17944e+01
102	-1.42289e-03	9.13011e-05	2.21752e-02	3.14049e+01
517	-5.96831e-05	8.03172e-06	4.43501e-04	6.99561e+01
950	-1.84349e-05	3.26717e-06	1.04018e-04	9.34893e+01
4800	-8.04548e-07	2.86464e-07	2.25961e-06	2.10639e+02
9750	-1.33337e-07	9.91180e-08	1.79370e-07	3.00488e+02
49238	1.69527e-08	8.73393e-09	3.29055e-08	6.73612e+02
100000	1.82845e-09	3.00705e-09	1.11179e-09	9.61804e+02

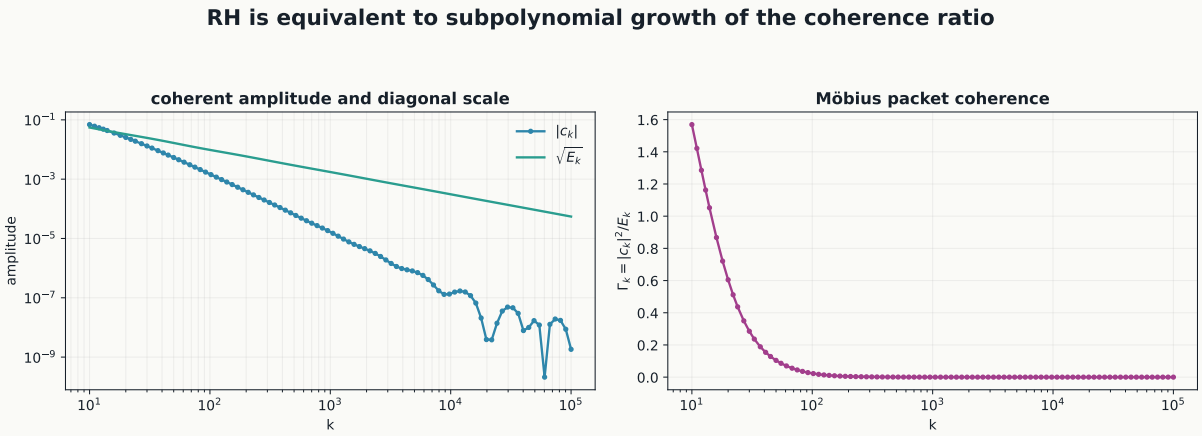


Figure 16.1: The principal transfer amplitude, diagonal energy scale, and coherence ratio. The numerical values are finite diagnostics only. They do not establish the required asymptotic bound.

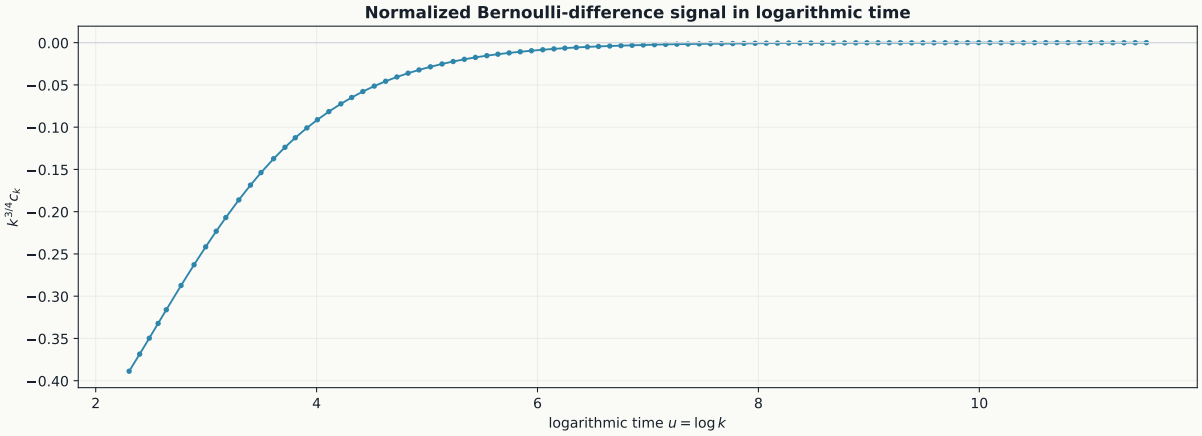


Figure 16.2: The normalized sequence $k^{3/4}c_k$ in logarithmic time. Zeros off the critical line would appear as expanding modes in this coordinate.

Boundary statement 16.2 (Why a generic frame estimate cannot prove RH). If every active packet had the same sign, the coherent-to-diagonal ratio would grow like $\sqrt{k}$. The missing amplitude factor is $k^{1/4}$. Therefore Cauchy-Schwarz, Parseval, positivity, or a sign-free frame bound cannot prove RH. Any successful estimate must use arithmetic structure of $\mu(n)$.

Chapter 17

Long-time mean power and a quantum correlation

Define

$$F_k(t) = \sum_{n=1}^{\infty} \frac{\mu(n)}{n^{2+\mathrm{i}t}} \left(1 - \frac{1}{n^2}\right)^k.$$

Then

$$F_k(0) = c_k.$$

Theorem 17.1 (Long-time diagonalization)**.**

$$\boxed{\lim_{T\to\infty} \frac{1}{2T} \int_{-T}^{T} |F_k(t)|^2 \, \mathrm{d}t = E_k.}$$

Proof. The frequencies $\log n$ are distinct. For a finite truncation, every off-diagonal time average tends to zero. The coefficient sequence is square summable, so the truncation tail is controlled in mean square. Pass to the limit. □

Therefore

$$\boxed{\Gamma_k = \frac{|F_k(0)|^2}{\lim_{T\to\infty} \frac{1}{2T} \int_{-T}^{T} |F_k(t)|^2 \, \mathrm{d}t}.}$$

RH says that the zero-time recurrence is only subpolynomially larger than the long-time mean power.

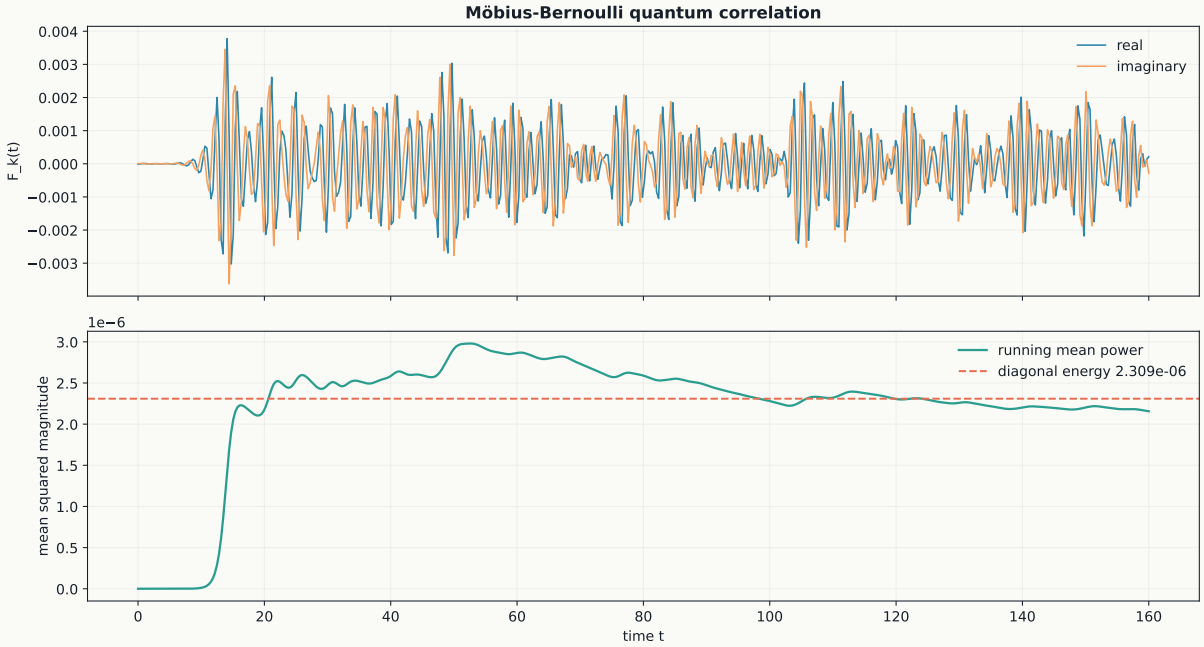


Figure 17.1: One finite Möbius-Bernoulli correlation and its running mean power. The dashed line is the exact diagonal energy of the truncated packet family.

For $a > 1$ and $k \geq 1$, work in $\ell^2(\{2, 3, \ldots\})$. Define

$$|\phi_a\rangle = \sum_{n \geq 2} n^{-a/2} |n\rangle,$$

$$H|n\rangle = \log n |n\rangle,$$
$$K_\beta |n\rangle = -\log(1 - n^{-\beta})|n\rangle,$$

and

$$\mathcal{M}_\chi |n\rangle = \mu(n)\chi(n)|n\rangle.$$

Then

$$\boxed{\mathcal{C}_{a,\beta,\chi}(k,t) = \langle \phi_a | \mathcal{M}_\chi \mathrm{e}^{-kK_\beta} \mathrm{e}^{-\mathrm{i}tH} | \phi_a \rangle.}$$

The logarithmic Hamiltonian H is the same spectral object used in the recent zeta-governed quantum-dynamics construction of Wei and collaborators [7]. That work builds thermal weights and phases from the spectrum $\log n$. The new ingredients here are the Möbius-character observable and the scale filter e^{-kK_β}. A complete gate construction would require a reversible Möbius or squarefree-phase oracle, which remains an engineering and complexity question.

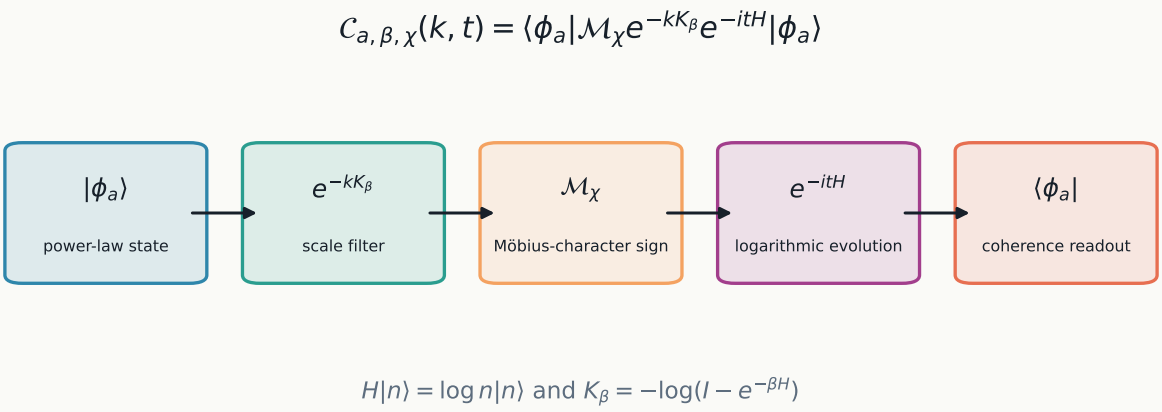


Figure 17.2: Operator realization of the generalized transfer function. Scale evolution and real-time logarithmic evolution commute because both are diagonal in the integer basis.

Chapter 18

Arithmetic polyphase and character packets

For a dyadic scale shell $I_J = [2^J, 2^{J+1})$ and a residue class $r \pmod M$, define

$$\boxed{F_{k;J,M,r}(t) = \sum_{\substack{n\in I_J \\ n\equiv r \pmod M}} \frac{\mu(n)}{n^{2+it}} \left(1 - \frac{1}{n^2}\right)^k.}$$

Then

$$F_k(t) = \sum_J \sum_{r=0}^{M-1} F_{k;J,M,r}(t).$$

Arithmetic refinement is exact:

$$\boxed{F_{k;J,M,r} = \sum_{\ell=0}^{L-1} F_{k;J,LM,r+\ell M}.}$$

Scale refinement is exact because each shell splits into two half-shells. The diagonal energy also separates exactly over every complete packet family.

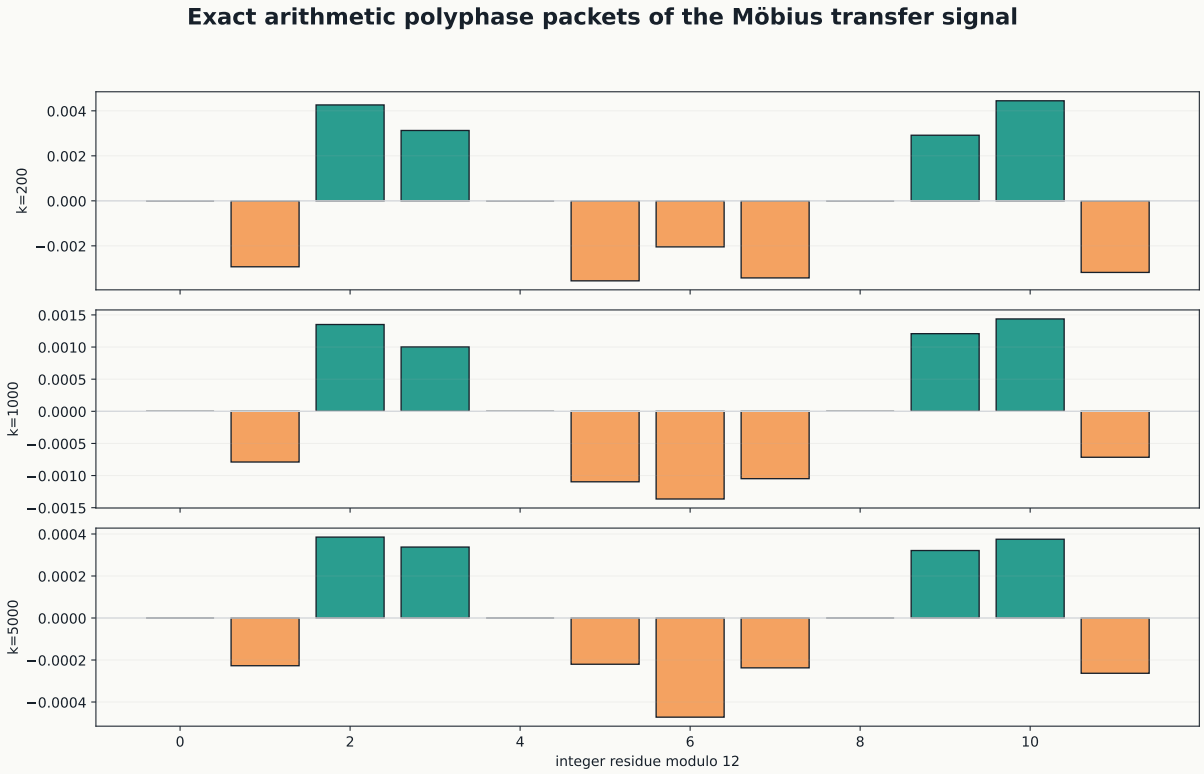


Figure 18.1: Residue packets of the principal Möbius transfer at three scales. Positive and negative packets shift with k, while their exact sum remains c_k.

For a Dirichlet character χ, the character transfer

$$\mathcal{C}_{a,\beta,\chi}(k,t) = \sum_{n\geq 1} \frac{\mu(n)\chi(n)}{n^{a+\mathrm{i}t}}(1-n^{-\beta})^k$$

provides a multiplicative-frequency coordinate. Known Riesz-type theorems give GRH criteria for continuous exponential kernels and related L-function classes [5, 6]. The corresponding discrete coherence criterion is a natural target, but no character-uniform theorem is claimed here.

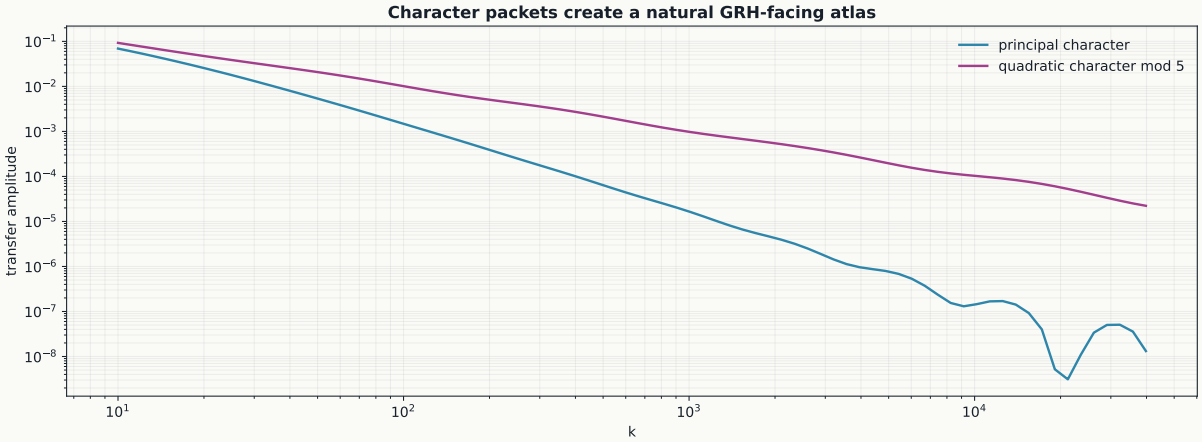


Figure 18.2: Principal and quadratic-character transfer amplitudes. The character coordinate can strongly alter cancellation even when the same scale filter is used.

Part IV

Worked computational atlas and research program

Chapter 19

Worked example: complete three-channel exponential lift

Start from

$$e^z = \sum_{n\geq 0} \frac{z^n}{n!}.$$

Separate the exponents modulo three:

$$E_{3,0}(u) = 1 + \frac{u}{3!} + \frac{u^2}{6!} + \cdots,$$
$$E_{3,1}(u) = 1 + \frac{u}{4!} + \frac{u^2}{7!} + \cdots,$$
$$E_{3,2}(u) = \frac{1}{2!} + \frac{u}{5!} + \frac{u^2}{8!} + \cdots.$$

The packet system is

$$\Theta_3 \begin{pmatrix} E_{3,0} \\ E_{3,1} \\ E_{3,2} \end{pmatrix} = \begin{pmatrix} 0 & 0 & u \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} E_{3,0} \\ E_{3,1} \\ E_{3,2} \end{pmatrix}.$$

Writing the rows explicitly gives

$$3uE'_{3,0} = uE_{3,2},$$
$$(3u\partial_u + 1)E_{3,1} = E_{3,0},$$
$$(3u\partial_u + 2)E_{3,2} = E_{3,1}.$$

Reconstruction yields

$$e^z = E_{3,0}(z^3) + zE_{3,1}(z^3) + z^2E_{3,2}(z^3).$$

Every equation can be checked coefficient by coefficient. The matrix system is not a numerical approximation. It is the scalar equation in an exact residue coordinate system.

Chapter 20

Worked example: Bernoulli packet refinement

Take $a = 1/3$, $M = 2$, and refine by $L = 2$. The parent packet is

$$\mathfrak{B}_{2,1}(z; 1/3) = -z2^{z-1}\zeta\left(1 - z, \frac{2}{3}\right).$$

Its two children are

$$\mathfrak{B}_{4,1}(z; 1/3) = -z4^{z-1}\zeta\left(1 - z, \frac{1}{3}\right)$$

and

$$\mathfrak{B}_{4,3}(z; 1/3) = -z4^{z-1}\zeta\left(1 - z, \frac{5}{6}\right).$$

The multiplication formula at modulus two gives

$$\zeta\left(1 - z, \frac{1}{3}\right) + \zeta\left(1 - z, \frac{5}{6}\right) = 2^{1-z}\zeta\left(1 - z, \frac{2}{3}\right).$$

Multiplication by $-z4^{z-1}$ gives

$$\boxed{\mathfrak{B}_{2,1}(z; 1/3) = \mathfrak{B}_{4,1}(z; 1/3) + \mathfrak{B}_{4,3}(z; 1/3).}$$

At $z = 6$, this becomes an exact identity among rational Bernoulli-polynomial values.

Chapter 21

Worked example: third-root cyclotomic norm

For $n = 8$, the three integer child packets at $a = 0$ are

$$-\frac{729}{10}, \qquad \frac{1093}{30}, \qquad \frac{1093}{30}.$$

Let $\omega^2 + \omega + 1 = 0$. The first non-DC detail is

$$D_1 = -\frac{729}{10} + \omega^2\frac{1093}{30} + \omega\frac{1093}{30} = -\frac{328}{3}.$$

Its cyclotomic norm is

$$\boxed{N_8 = D_1\overline{D_1} = \frac{107584}{9} = \left(\frac{328}{3}\right)^2.}$$

For odd $n \geq 3$, the same calculation produces three times a rational square. The table in table 10.1 records the first values.

Chapter 22

Worked example: the principal transfer at order 1200

For $k = 1200$, define

$$w_n = \frac{\mu(n)}{n^2}\left(1 - \frac{1}{n^2}\right)^{1200}.$$

The finite computation to $n = 200000$ gives a zero-time amplitude of approximately

$$|F_{1200}(0)| = 1.1153 \times 10^{-5}.$$

The diagonal packet power is approximately

$$E_{1200} = 2.3091 \times 10^{-6}.$$

Hence

$$\Gamma_{1200} \approx 5.3868 \times 10^{-5}.$$

This is one finite numerical value. It is not an asymptotic theorem. Its purpose is to show how the exact definitions become a measurable signal-to-energy ratio.

Chapter 23

Algorithmic workflow

Algorithm 23.1 (Generating a polyphase special-function system)**.** Given a scalar power series or differential equation:

1. Choose a modulus M and separate Taylor exponents by residue.
2. Form $\mathbf{F}_M(u)$.
3. Replace multiplication by z with $C_M(u)$.
4. Replace $z\frac{\mathrm{d}}{\mathrm{d}z}$ with Θ_M.
5. Preserve operator order and derive the matrix system.
6. If a parent-child identity is available, refine packet addresses.
7. Apply a unitary transform to create low-pass and detail channels.
8. Derive inverse synthesis and energy identities.
9. Build Wronskians, recurrences, arithmetic traces, or coherence ratios suited to the question.
10. Test simple conjectures against exact and high-precision counterexamples before promoting them.

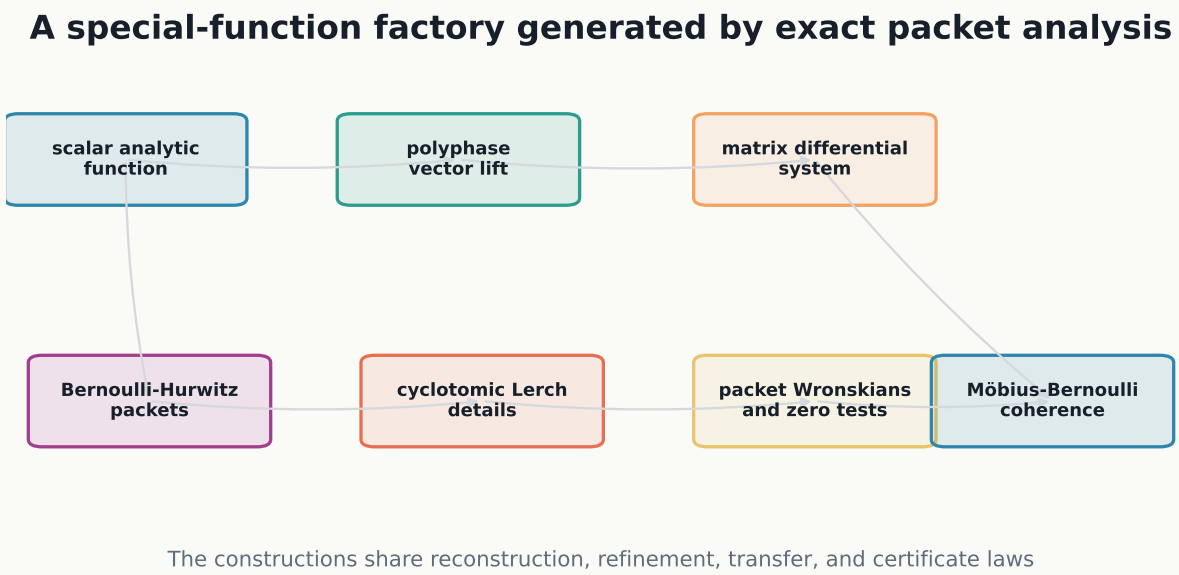


Figure 23.1: The generated function families and their main transfer routes. The same packet operations produce differential systems, refinable Bernoulli packets, cyclotomic details, Wronskians, and arithmetic coherence signals.

Chapter 24

Open problems generated by the calculus

24.1 Near-term problems

P1. Determine sharp denominator and local valuation laws for cyclotomic Bernoulli details at rational a.

P2. Classify the orders and parameter intervals for which selected packet Wronskians have one sign.

P3. Determine when Bernoulli and Euler channel zeros interlace after completion or weighting.

P4. Develop dominant-pole asymptotics for reciprocal and logarithmic derivatives of non-DC detail functions.

P5. Find minimal matrix differential systems after symmetry reduction of the general polyphase lift.

P6. Identify transforms for which packet children form a positive or totally positive kernel.

24.2 Major-conjecture interfaces

M1. Prove subpolynomial Möbius packet coherence in the principal zeta case.

M2. Establish a character-uniform version compatible with GRH criteria.

M3. Construct positive canonical systems from packet Gram matrices.

M4. Connect local p-adic packet estimates to Archimedean coherence through a quantitative global height theorem.

Conjecture 24.1 (Möbius packet coherence)**.** *For every $\varepsilon > 0$,*

$$\boxed{\left|\sum_{n=1}^{\infty} \frac{\mu(n)}{n^2}\left(1-\frac{1}{n^2}\right)^k\right|^2 \le C_\varepsilon k^\varepsilon \sum_{n=1}^{\infty} \frac{\mu(n)^2}{n^4}\left(1-\frac{1}{n^2}\right)^{2k}.}$$

By the proved energy asymptotic and the Báez-Duarte criterion, this conjecture is equivalent to RH. It is included as the sharp theorem target, not as a claimed result.

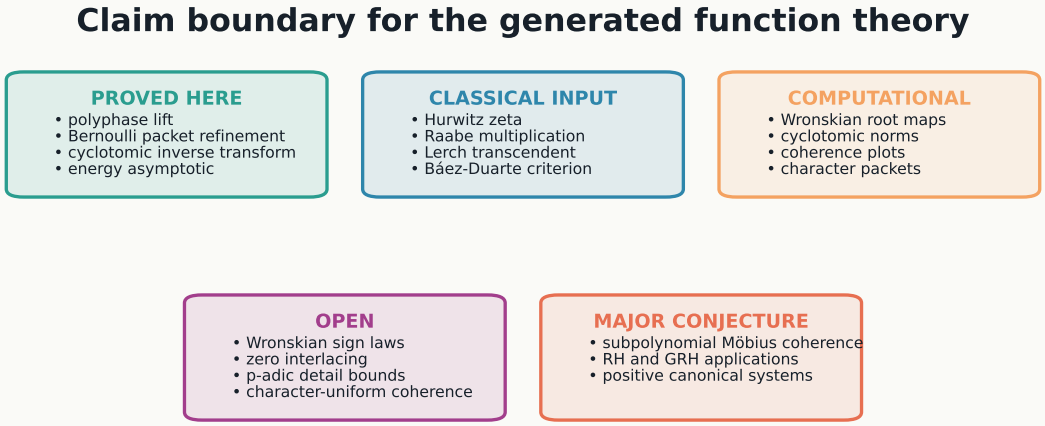


Figure 24.1: Status map for the paper. The map separates proved identities, classical inputs, finite computations, open special-function questions, and major conjecture interfaces.

Chapter 25

Verification

Every figure and numerical table was generated by the accompanying Python source. The symbolic suite verifies coefficient reconstruction, the Weyl commutator, hypergeometric exponential packets, Bernoulli and Euler channel identities, complex-order Bernoulli reconstruction and refinement, Lerch closed forms, and packet Wronskian formulas.

The asymptotic plots use a Möbius sieve through 200000. The largest plotted transfer order is 100000. These computations illustrate the proved formulas and expose possible failure modes. They do not replace the asymptotic theorems or the open coherence estimate.

Table 25.1: Exact and high-precision verification summary.

Check family	checks	passed
Bernoulli	15	15
Euler	9	9
Lerch	3	3
Weyl	4	4
exponential	9	9
packet	7	7
polynomial	4	4
Total	51	51

Chapter 26

Conclusion

The calculus generates a coherent special-function theory rather than a list of renamed scalar functions. The defining structure is:

analysis $\longrightarrow$ matrix transfer $\longrightarrow$ refinement $\longrightarrow$ unitary detail transform $\longrightarrow$ synthesis $\longrightarrow$ certificate.

The general polyphase lift transports scalar differential equations into exact matrix systems. Bernoulli-Hurwitz packets extend Raabe multiplication to complex order. Their cyclotomic transform produces Lerch details with perfect reconstruction. The two-channel case unifies Bernoulli and Euler polynomials. Packet Wronskians create direct zero diagnostics and also reveal false broad conjectures. The Möbius-Bernoulli transfer family turns reciprocal L-values into scale-localized signals whose diagonal energy is explicit. In the zeta specialization, RH is precisely the remaining coherence estimate.

The practical next step is not to introduce more notation. It is to prove one nontrivial property of a generated family that is not automatic from the scalar parent. The most accessible targets are denominator laws, Wronskian signs, symmetry reductions, and reciprocal-detail recurrences. The most difficult target is the Möbius coherence inequality.

Chapter A

Formula registry

Object	Defining formula
Polyphase section	$F_{M,r}(u) = \sum_{m\ge 0} a_{Mm+r} u^m$
Carry shift	$C_M(u)_{0,M-1} = u$, $C_M(u)_{r,r-1} = 1$
Packet Euler operator	$\Theta_M = Mu\partial_u I + \operatorname{diag}(0,\dots,M-1)$
Bernoulli interpolation	$\mathfrak{B}(z;a) = -z\zeta(1-z,a)$
Bernoulli packet	$\mathfrak{B}_{M,r}(z;a) = -zM^{z-1}\zeta(1-z,(a+r)/M)$
Cyclotomic detail	$\mathcal{W}_{M,r}^{(L,j)} = L^{-1/2}\sum_\ell \omega_L^{-j\ell}\mathfrak{B}_{LM,r+\ell M}$
Cyclotomic EGF	$\sum_n W_{L,j}(n;a)t^n/n! = te^{at}/[\sqrt{L}(\omega_L^{-j}e^t-1)]$
Packet Wronskian	$\mathcal{D}_{j,\ell} = \mathcal{W}_j(z)\mathcal{W}_\ell(z-1) - \mathcal{W}_\ell(z)\mathcal{W}_j(z-1)$
General transfer	$\mathcal{C}_{a,\beta,\chi}(k,t) = \sum_{j=0}^k (-1)^j \binom{k}{j} / L(a+\beta j+\mathrm{i}t,\chi)$

Object	Defining formula
Möbius form	$\mathcal{C} = \sum_n \mu(n)\chi(n)n^{-a-\mathrm{i}t}(1-n^{-\beta})^k$
Continuous companion	$\mathcal{R} = x\sum_n \mu(n)\chi(n)n^{-a-\mathrm{i}t}\mathrm{e}^{-x/n^\beta}$
Packet energy	$E_{a,\beta,\chi} = \sum_n \mu(n)^2\lvert\chi(n)\rvert^2 n^{-2a}(1-n^{-\beta})^{2k}$
Coherence ratio	$\Gamma = \lvert\mathcal{C}\rvert^2/E$

Chapter B

Complete Python source

The following program generates every figure, table, and numerical record used in this paper.

```
from __future__ import annotations

import argparse
import cmath
import csv
import hashlib
import json
import math
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable, Sequence

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Rectangle
import mpmath as mp
import numpy as np
import sympy as sp
from scipy.special import gamma

mp.mp.dps = 40

ROOT = Path(__file__).resolve().parent
FIGURES = ROOT / "figures"
RESULTS = ROOT / "results"
TABLES = ROOT / "tables"

for directory in (FIGURES, RESULTS, TABLES):
    directory.mkdir(parents=True, exist_ok=True)

INK = "#17202A"
BLUE = "#2E86AB"
TEAL = "#2A9D8F"
ORANGE = "#F4A261"
MAGENTA = "#A23E8C"
RED = "#E76F51"
YELLOW = "#E9C46A"
MUTED = "#5D6D7E"
PAPER = "#FAFAF7"
GRID = "#D5D8DC"
COLORS = [BLUE, TEAL, ORANGE, MAGENTA, RED, YELLOW]

plt.rcParams.update(
    {
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
        "xtick.color": INK,
        "ytick.color": INK,
```

```
        "text.color": INK,
        "font.size": 10.5,
        "axes.titlesize": 12.5,
        "axes.titleweight": "bold",
        "legend.frameon": False,
        "savefig.bbox": "tight",
        "savefig.facecolor": PAPER,
    }
)


def save_figure(fig: plt.Figure, stem: str) -> None:
    fig.savefig(FIGURES / f"{stem}.pdf")
    fig.savefig(FIGURES / f"{stem}.png", dpi=220)
    plt.close(fig)


def complex_array(values: Iterable[complex]) -> np.ndarray:
    return np.asarray([complex(value) for value in values], dtype=np.complex128)


def mobius_sieve(nmax: int) -> np.ndarray:
    """Linear sieve for the Mobius function on 0..nmax."""
    mu = np.zeros(nmax + 1, dtype=np.int8)
    mu[1] = 1
    primes: list[int] = []
    composite = np.zeros(nmax + 1, dtype=bool)
    for i in range(2, nmax + 1):
        if not composite[i]:
            primes.append(i)
            mu[i] = -1
        for p in primes:
            ip = i * p
            if ip > nmax:
                break
            composite[ip] = True
            if i % p == 0:
                mu[ip] = 0
                break
            mu[ip] = -mu[i]
    return mu


# ---------------------------------------------------------------------------
# General polyphase lift
# ---------------------------------------------------------------------------


def carry_matrix_symbolic(m: int, u: sp.Symbol) -> sp.Matrix:
    matrix = sp.zeros(m)
    matrix[0, m - 1] = u
    for row in range(1, m):
        matrix[row, row - 1] = 1
    return matrix


def polyphase_sections_from_coefficients(coeffs: Sequence[sp.Expr], m: int, u: sp.Symbol) -> sp.Matrix:
    sections: list[sp.Expr] = []
    for r in range(m):
        section = sp.Integer(0)
        q = 0
        index = r
        while index < len(coeffs):
            section += coeffs[index] * u**q
            q += 1
            index += m
        sections.append(sp.expand(section))
    return sp.Matrix(sections)


def reconstruct_sections(sections: Sequence[sp.Expr], m: int, z: sp.Symbol) -> sp.Expr:
```

```
    u_value = z**m
    return sp.expand(sum(z**r * sections[r].subs({sp.Symbol("u"): u_value}) for r in range(m)))


def theta_apply(section: sp.Expr, r: int, m: int, u: sp.Symbol) -> sp.Expr:
    return sp.expand(m * u * sp.diff(section, u) + r * section)


def theta_vector_apply(vector: sp.Matrix, m: int, u: sp.Symbol) -> sp.Matrix:
    return sp.Matrix([theta_apply(vector[r], r, m, u) for r in range(m)])


def exp_section_hypergeometric(m: int, r: int, u: complex) -> complex:
    params = [mp.mpf(r + j) / m for j in range(1, m + 1)]
    params = [value for value in params if abs(value - 1) > mp.mpf("1e-40")]
    return mp.hyper([], params, u / (m**m)) / mp.factorial(r)


def exp_section_series(m: int, r: int, u: complex, terms: int = 80) -> complex:
    return mp.nsum(lambda q: u**q / mp.factorial(m * q + r), [0, mp.inf]) if terms <= 0 else sum(
        mp.mpf(u) ** q / mp.factorial(m * q + r) for q in range(terms)
    )


# ---------------------------------------------------------------------------
# Bernoulli-Hurwitz packet functions and cyclotomic details
# ---------------------------------------------------------------------------


def bernoulli_interpolation(z: complex, a: complex = 1.0) -> complex:
    if abs(z) < 1e-20:
        return 1.0 + 0.0j
    zz = mp.mpc(z)
    return complex(-zz * mp.zeta(1 - zz, a))


def bernoulli_packet(z: complex, a: complex, m: int, r: int) -> complex:
    if abs(z) < 1e-20:
        return 1.0 / m
    argument = (a + r) / m
    zz = mp.mpc(z)
    return complex(-zz * (m ** (zz - 1)) * mp.zeta(1 - zz, argument))


def cyclotomic_detail(z: complex, a: complex, m: int, r: int, ell: int, j: int) -> complex:
    omega = cmath.exp(2j * math.pi / ell)
    values = [bernoulli_packet(z, a, m * ell, r + child * m) for child in range(ell)]
    return sum((omega ** (-j * child)) * values[child] for child in range(ell)) / math.sqrt(ell)


def cyclotomic_detail_lerch(z: complex, a: complex, m: int, r: int, ell: int, j: int) -> complex:
    omega = mp.e ** (2j * mp.pi / ell)
    argument = (a + r) / m
    return complex(
        -mp.mpc(z)
        / mp.sqrt(ell)
        * (m ** (mp.mpc(z) - 1))
        * mp.lerchphi(omega ** (-j), 1 - mp.mpc(z), argument)
    )


def cyclotomic_egf(t: complex, a: complex, ell: int, j: int) -> complex:
    omega = cmath.exp(2j * math.pi / ell)
    denominator = omega ** (-j) * cmath.exp(t) - 1
    if abs(t) < 1e-14 and j == 0:
        return 1 / math.sqrt(ell)
    return t * cmath.exp(a * t) / (math.sqrt(ell) * denominator)


def bernoulli_euler_channels(n: int, a: sp.Expr) -> tuple[sp.Expr, sp.Expr]:
    low = sp.bernoulli(n, a) / sp.sqrt(2)
```

```
    if n == 0:
        high = sp.Integer(0)
    else:
        high = -sp.Rational(n, 2) * sp.euler(n - 1, a) / sp.sqrt(2)
    return sp.simplify(low), sp.simplify(high)


def packet_wronskian_polynomial(n: int, x: sp.Symbol) -> sp.Expr:
    if n < 2:
        raise ValueError("n must be at least 2")
    return sp.expand(
        n * sp.bernoulli(n - 1, x) * sp.euler(n - 1, x)
        - (n - 1) * sp.bernoulli(n, x) * sp.euler(n - 2, x)
    )


# ---------------------------------------------------------------------------
# Mobius-Bernoulli transfer family
# ---------------------------------------------------------------------------


def principal_transfer_values(
    k_values: Sequence[int],
    nmax: int,
    mu: np.ndarray | None = None,
) -> dict[str, np.ndarray]:
    if mu is None:
        mu = mobius_sieve(nmax)
    n = np.arange(2, nmax + 1, dtype=np.float64)
    mu_n = mu[2:].astype(np.float64)
    log_lambda = np.log1p(-1.0 / (n * n))
    base = 1.0 / (n * n)
    squarefree = mu_n * mu_n
    c_values = []
    energy_values = []
    mass_values = []
    for k in k_values:
        weight = base * np.exp(k * log_lambda)
        c_values.append(float(np.dot(mu_n, weight)))
        energy_values.append(float(np.dot(squarefree, weight * weight)))
        mass_values.append(float(np.dot(squarefree, weight)))
    c = np.asarray(c_values)
    energy = np.asarray(energy_values)
    mass = np.asarray(mass_values)
    coherence = np.divide(c * c, energy, out=np.zeros_like(c), where=energy > 0)
    effective_dimension = np.divide(mass * mass, energy, out=np.zeros_like(mass), where=energy > 0)
    return {
        "k": np.asarray(k_values, dtype=int),
        "c": c,
        "energy": energy,
        "mass": mass,
        "coherence": coherence,
        "effective_dimension": effective_dimension,
    }


def generalized_transfer_direct(
    a: float,
    beta: float,
    k: int,
    t: float,
    nmax: int,
    mu: np.ndarray,
    character: np.ndarray | None = None,
) -> complex:
    n = np.arange(2, nmax + 1, dtype=np.float64)
    coefficients = mu[2 : nmax + 1].astype(np.complex128)
    if character is not None:
        coefficients = coefficients * character[2 : nmax + 1]
    damping = np.exp(k * np.log1p(-1.0 / np.power(n, beta)))
    phase = np.exp(-1j * t * np.log(n))
```

```
    total = np.sum(coefficients * phase * damping / np.power(n, a))
    if k == 0:
        total += 1.0
    return complex(total)


def long_time_power(
    a: float,
    beta: float,
    k: int,
    nmax: int,
    mu: np.ndarray,
    character: np.ndarray | None = None,
) -> float:
    n = np.arange(2, nmax + 1, dtype=np.float64)
    coefficients = mu[2 : nmax + 1].astype(np.complex128)
    if character is not None:
        coefficients = coefficients * character[2 : nmax + 1]
    damping = np.exp(k * np.log1p(-1.0 / np.power(n, beta)))
    values = coefficients * damping / np.power(n, a)
    total = np.sum(np.abs(values) ** 2)
    if k == 0:
        total += 1.0
    return float(total)


def quadratic_character_mod5(nmax: int) -> np.ndarray:
    values = np.zeros(nmax + 1, dtype=np.int8)
    residue_value = {0: 0, 1: 1, 2: -1, 3: -1, 4: 1}
    for n in range(1, nmax + 1):
        values[n] = residue_value[n % 5]
    return values.astype(np.complex128)


def polyphase_transfer_packets(k: int, modulus: int, nmax: int, mu: np.ndarray) -> np.ndarray:
    n = np.arange(2, nmax + 1, dtype=np.float64)
    weights = mu[2 : nmax + 1].astype(np.float64) / (n * n) * np.exp(k * np.log1p(-1.0 / (n * n)))
    packets = np.zeros(modulus, dtype=np.float64)
    residues = np.arange(2, nmax + 1) % modulus
    for residue in range(modulus):
        packets[residue] = float(np.sum(weights[residues == residue]))
    return packets


# ---------------------------------------------------------------------------
# Exact checks and tables
# ---------------------------------------------------------------------------


@dataclass
class Check:
    name: str
    passed: bool
    detail: str


def run_symbolic_checks() -> list[Check]:
    checks: list[Check] = []
    z, u = sp.symbols("z u")

    coeffs = [sp.Rational((index + 1) * (-1) ** index, index + 2) for index in range(15)]
    for m in (2, 3, 4, 5):
        sections = polyphase_sections_from_coefficients(coeffs, m, u)
        reconstruction = sp.expand(sum(z**r * sections[r].subs(u, z**m) for r in range(m)))
        original = sp.expand(sum(coeffs[index] * z**index for index in range(len(coeffs))))
        checks.append(Check(f"polynomial reconstruction M={m}", sp.simplify(reconstruction - original) == 0, "exact"
    ))

        carry = carry_matrix_symbolic(m, u)
        theta_vector = theta_vector_apply(sections, m, u)
        theta_carry = theta_vector_apply(carry * sections, m, u)
```

```
        carry_theta = carry * theta_vector
        commutator = sp.simplify(theta_carry - carry_theta - carry * sections)
        checks.append(Check(f"Weyl commutator M={m}", all(value == 0 for value in commutator), "[Theta,C]=C"))

    for m in (2, 3, 4):
        for r in range(m):
            point = mp.mpf("0.37")
            series_value = exp_section_series(m, r, point, 80)
            hyper_value = exp_section_hypergeometric(m, r, point)
            residual = abs(series_value - hyper_value)
            checks.append(Check(f"exponential hypergeometric M={m} r={r}", residual < mp.mpf("1e-35"), f"{float(
     residual):.3e}"))

    a = sp.symbols("a")
    for n in range(1, 10):
        child0 = 2 ** (n - 1) * sp.bernoulli(n, a / 2)
        child1 = 2 ** (n - 1) * sp.bernoulli(n, (a + 1) / 2)
        low, high = bernoulli_euler_channels(n, a)
        low_direct = sp.simplify((child0 + child1) / sp.sqrt(2))
        high_direct = sp.simplify((child0 - child1) / sp.sqrt(2))
        checks.append(Check(f"Bernoulli low channel n={n}", sp.simplify(low_direct - low) == 0, "exact"))
        checks.append(Check(f"Euler high channel n={n}", sp.simplify(high_direct - high) == 0, "exact"))

    z0 = 0.37 + 0.41j
    a0 = 0.62
    for m in (2, 3, 4):
        reconstructed = sum(bernoulli_packet(z0, a0, m, r) for r in range(m))
        target = bernoulli_interpolation(z0, a0)
        residual = abs(reconstructed - target)
        checks.append(Check(f"Bernoulli packet reconstruction M={m}", residual < 1e-11, f"{residual:.3e}"))

    for m, ell, r in ((1, 2, 0), (2, 3, 1), (3, 2, 2)):
        parent = bernoulli_packet(z0, a0, m, r)
        children = sum(bernoulli_packet(z0, a0, m * ell, r + child * m) for child in range(ell))
        residual = abs(parent - children)
        checks.append(Check(f"Bernoulli refinement M={m} L={ell} r={r}", residual < 1e-11, f"{residual:.3e}"))

    for m, r, ell, j in ((1, 0, 3, 1), (2, 1, 4, 2), (3, 0, 2, 1)):
        direct = cyclotomic_detail(z0, a0, m, r, ell, j)
        lerch = cyclotomic_detail_lerch(z0, a0, m, r, ell, j)
        residual = abs(direct - lerch)
        checks.append(Check(f"Lerch detail M={m} L={ell} j={j}", residual < 1e-10, f"{residual:.3e}"))

    x = sp.symbols("x")
    for n in range(2, 9):
        polynomial = packet_wronskian_polynomial(n, x)
        low_n, high_n = bernoulli_euler_channels(n, x)
        low_prev, high_prev = bernoulli_euler_channels(n - 1, x)
        direct = sp.expand(4 * (low_n * high_prev - high_n * low_prev))
        checks.append(Check(f"packet Wronskian n={n}", sp.simplify(polynomial - direct) == 0, "exact"))

    return checks


def write_cyclotomic_norm_table() -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []
    for n in range(1, 15):
        children = [sp.simplify(3 ** (n - 1) * sp.bernoulli(n, sp.Rational(r, 3))) for r in range(3)]
        # D_1 = child0 + omega^2 child1 + omega child2 = A + B omega, omega^2+omega+1=0
        a_part = sp.simplify(children[0] - children[1])
        b_part = sp.simplify(-children[1] + children[2])
        norm = sp.factor(a_part**2 - a_part * b_part + b_part**2)
        rows.append(
            {
                "n": str(n),
                "child_0": str(children[0]),
                "child_1": str(children[1]),
                "child_2": str(children[2]),
                "A": str(a_part),
                "B": str(b_part),
                "norm": str(norm),
```

```
            }
        )
    with (TABLES / "cyclotomic_norms_L3.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
    return rows


# ---------------------------------------------------------------------------
# Figures
# ---------------------------------------------------------------------------


def figure_01_polyphase_partition() -> None:
    m = 4
    degree = 31
    fig, ax = plt.subplots(figsize=(12.8, 4.8))
    for n in range(degree + 1):
        residue = n % m
        ax.add_patch(Rectangle((n - 0.42, residue - 0.32), 0.84, 0.64, facecolor=COLORS[residue] + "88", edgecolor=INK, lw=0.6))
        ax.text(n, residue, str(n), ha="center", va="center", fontsize=7.5)
    ax.set_xlim(-1, degree + 1)
    ax.set_ylim(-0.8, m - 0.2)
    ax.set_yticks(range(m), [f"packet r={r}" for r in range(m)])
    ax.set_xlabel("Taylor exponent n")
    ax.set_title("Every coefficient exponent has one polyphase address")
    ax.grid(False)
    fig.tight_layout()
    save_figure(fig, "01_polyphase_partition")


def figure_02_carry_shift() -> None:
    m = 5
    fig, ax = plt.subplots(figsize=(12.8, 5.7))
    ax.axis("off")
    y = 0.58
    xs = np.linspace(0.10, 0.90, m)
    for r, xpos in enumerate(xs):
        box = FancyBboxPatch((xpos - 0.065, y - 0.08), 0.13, 0.16, boxstyle="round,pad=.01", facecolor=COLORS[r % len(COLORS)] + "28", edgecolor=COLORS[r % len(COLORS)], lw=2)
        ax.add_patch(box)
        ax.text(xpos, y, rf"$F_{{{r}}}(u)$", ha="center", va="center", fontsize=15)
    for r in range(m - 1):
        arrow = FancyArrowPatch((xs[r] + 0.066, y), (xs[r + 1] - 0.066, y), arrowstyle="-|>", mutation_scale=13, lw=1.8, color=INK)
        ax.add_patch(arrow)
        ax.text((xs[r] + xs[r + 1]) / 2, y + 0.055, r"multiply by $z$", ha="center", fontsize=9)
    arrow = FancyArrowPatch((xs[-1], y - 0.085), (xs[0], y - 0.085), connectionstyle="arc3,rad=-0.47", arrowstyle="-|>", mutation_scale=13, lw=2.2, color=MAGENTA)
    ax.add_patch(arrow)
    ax.text(0.5, 0.15, r"wraparound contributes $uF_{M-1}(u)$ to packet $0$", ha="center", fontsize=13, color=MAGENTA)
    ax.text(0.5, 0.89, r"The carry-shift matrix $C_M(u)$ is multiplication by $z$ in packet coordinates", ha="center", fontsize=18, weight="bold")
    save_figure(fig, "02_carry_shift")


def figure_03_exp_sections() -> None:
    u = np.linspace(0, 8, 350)
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
    for m, ax in zip((2, 3), axes, strict=True):
        for r in range(m):
            values = [float(mp.re(exp_section_hypergeometric(m, r, value))) for value in u]
            ax.plot(u, values, lw=2, label=rf"$E_{{{m},{r}}}(u)$")
        ax.set_title(f"Exponential packets for M={m}")
        ax.set_xlabel("u")
        ax.set_ylabel("packet value")
        ax.grid(alpha=0.18)
        ax.legend()
```

```
    fig.suptitle("The scalar exponential becomes a vector hypergeometric function", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.92])
    save_figure(fig, "03_exponential_sections")


def figure_04_exp_reconstruction() -> None:
    z = np.linspace(-2.5, 2.5, 500)
    fig, ax = plt.subplots(figsize=(12.8, 4.8))
    target = np.exp(z)
    for m, color in zip((2, 3, 4), (BLUE, TEAL, MAGENTA), strict=True):
        reconstruction = np.zeros_like(z)
        for r in range(m):
            reconstruction += z**r * np.asarray([float(mp.re(exp_section_hypergeometric(m, r, value**m))) for value in z])
        ax.plot(z, reconstruction - target, color=color, label=f"M={m}")
    ax.set_yscale("symlog", linthresh=1e-15)
    ax.set_xlabel("z")
    ax.set_ylabel("reconstruction residual")
    ax.set_title(r"Exact synthesis $e^z=\sum_r z^r E_{M,r}(z^M)$")
    ax.grid(alpha=0.2)
    ax.legend()
    fig.tight_layout()
    save_figure(fig, "04_exponential_reconstruction")


def figure_05_operator_lift_map() -> None:
    fig, ax = plt.subplots(figsize=(12.8, 6.0))
    ax.axis("off")
    stages = [
        (0.04, "scalar function", r"$F(z)$", BLUE),
        (0.28, "residue analysis", r"$\mathbf{F}_M(u)$", TEAL),
        (0.53, "operator lift", r"$z\mapsto C_M$\n$\theta\mapsto\Theta_M$", ORANGE),
        (0.78, "matrix system", r"$\mathcal{L}(C_M,\Theta_M)\mathbf{F}_M=0$", MAGENTA),
    ]
    for x0, title, formula, color in stages:
        box = FancyBboxPatch((x0, 0.38), 0.18, 0.25, boxstyle="round,pad=.018", facecolor=color + "24", edgecolor=color, lw=2.2)
        ax.add_patch(box)
        ax.text(x0 + 0.09, 0.57, title, ha="center", va="center", fontsize=12, weight="bold")
        ax.text(x0 + 0.09, 0.46, formula, ha="center", va="center", fontsize=13)
    for left, right in zip(stages[:-1], stages[1:], strict=True):
        ax.add_patch(FancyArrowPatch((left[0] + 0.18, 0.505), (right[0], 0.505), arrowstyle="-|>", mutation_scale=15, lw=2, color=INK))
    ax.text(0.5, 0.83, "Polyphase lifting is a representation of the same Weyl algebra", ha="center", fontsize=19, weight="bold")
    ax.text(0.5, 0.17, r"The commutator is preserved: $[\Theta_M,C_M]=C_M$", ha="center", fontsize=15, color=MUTED)
    save_figure(fig, "05_operator_lift_map")


def figure_06_bessel_sections() -> None:
    z_values = np.linspace(-8, 8, 800)
    m = 3
    # J0 Taylor coefficients up to degree 72
    coeffs = [sp.Integer(0)] * 73
    for q in range(37):
        coeffs[2 * q] = sp.Rational((-1) ** q, 4**q * math.factorial(q) ** 2)
    u_symbol = sp.Symbol("u")
    sections = polyphase_sections_from_coefficients(coeffs, m, u_symbol)
    functions = [sp.lambdify(u_symbol, section, "numpy") for section in sections]
    fig, axes = plt.subplots(m, 1, figsize=(12.8, 7.2), sharex=True)
    for r, ax in enumerate(axes):
        u_values = z_values**m
        values = np.real(functions[r](u_values))
        ax.plot(z_values, values, color=COLORS[r], lw=1.6)
        ax.set_ylabel(rf"$J_{{0;3,{r}}}$")
        ax.grid(alpha=0.16)
    axes[-1].set_xlabel("z, with packet evaluated at u=z^3")
    fig.suptitle("Three packet sections of the Bessel function J0", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.94])
    save_figure(fig, "06_bessel_sections")

```

```

def figure_07_bernoulli_packet_reconstruction() -> None:
    a = 0.37
    z_values = np.linspace(-1.5, 2.5, 240)
    m = 4
    packets = np.zeros((m, len(z_values)), dtype=np.float64)
    total = np.zeros(len(z_values), dtype=np.float64)
    for index, z in enumerate(z_values):
        for r in range(m):
            packets[r, index] = bernoulli_packet(z, a, m, r).real
        total[index] = bernoulli_interpolation(z, a).real
    fig, axes = plt.subplots(2, 1, figsize=(12.8, 7.0), sharex=True)
    for r in range(m):
        axes[0].plot(z_values, packets[r], lw=1.5, label=f"r={r}")
    axes[0].set_ylabel("packet value")
    axes[0].set_title(r"Bernoulli-Hurwitz packets $\mathcal{B}_{4,r}(z;a)$")
    axes[0].legend(ncol=4)
    axes[0].grid(alpha=0.16)
    axes[1].plot(z_values, total, color=INK, lw=2.4, label="direct interpolation")
    axes[1].plot(z_values, packets.sum(axis=0), color=RED, lw=1.2, ls="--", label="packet sum")
    axes[1].set_xlabel("complex order z restricted to the real axis")
    axes[1].set_ylabel("reconstructed value")
    axes[1].grid(alpha=0.16)
    axes[1].legend()
    fig.suptitle("Exact reconstruction of the Bernoulli interpolation from four packets", fontsize=17, weight="bold"
    )
    fig.tight_layout(rect=[0, 0, 1, 0.94])
    save_figure(fig, "07_bernoulli_packet_reconstruction")


def figure_08_bernoulli_refinement_tree() -> None:
    fig, ax = plt.subplots(figsize=(12.8, 7.0))
    ax.axis("off")
    levels = [(1, [0]), (2, [0, 1]), (4, list(range(4))), (8, list(range(8)))]
    positions: dict[tuple[int, int], tuple[float, float]] = {}
    for depth, (modulus, residues) in enumerate(levels):
        y = 0.86 - depth * 0.23
        xs = np.linspace(0.08, 0.92, len(residues))
        for residue, xpos in zip(residues, xs, strict=True):
            positions[(modulus, residue)] = (xpos, y)
            color = COLORS[depth % len(COLORS)]
            ax.add_patch(FancyBboxPatch((xpos - 0.045, y - 0.032), 0.09, 0.064, boxstyle="round,pad=.005", facecolor
    =color + "25", edgecolor=color, lw=1.6))
            ax.text(xpos, y, rf"${{modulus}},{residue}}$", ha="center", va="center", fontsize=8.5)
    for (parent_m, residues), (child_m, child_residues) in zip(levels[:-1], levels[1:], strict=True):
        factor = child_m // parent_m
        for parent_r in residues:
            px, py = positions[(parent_m, parent_r)]
            for ell in range(factor):
                child_r = parent_r + ell * parent_m
                cx, cy = positions[(child_m, child_r)]
                ax.plot([px, cx], [py - 0.035, cy + 0.035], color=GRID, lw=1)
    ax.text(0.5, 0.96, r"Exact refinement: $\mathcal{B}_{M,r}=\sum_\ell \mathcal{B}_{LM,r+\ell M}$", ha="center",
    fontsize=19, weight="bold")
    ax.text(0.5, 0.04, "Each parent is reconstructed by all of its residue children", ha="center", fontsize=12.5,
    color=MUTED)
    save_figure(fig, "08_bernoulli_refinement_tree")


def figure_09_cyclotomic_details() -> None:
    z = 3.4 + 0.6j
    a_values = np.linspace(0.1, 1.8, 120)
    ell = 4
    fig, axes = plt.subplots(2, 1, figsize=(12.8, 7.0), sharex=True)
    for j in range(ell):
        values = np.asarray([cyclotomic_detail(z, a, 1, 0, ell, j) for a in a_values])
        axes[0].plot(a_values, np.abs(values), lw=1.8, label=f"j={j}")
        axes[1].plot(a_values, np.unwrap(np.angle(values)), lw=1.5, label=f"j={j}")
    axes[0].set_ylabel("magnitude")
    axes[0].set_title("Cyclotomic Bernoulli-Lerch channels")
    axes[0].grid(alpha=0.16)
```

```
    axes[0].legend(ncol=4)
    axes[1].set_xlabel("translation parameter a")
    axes[1].set_ylabel("unwrapped phase")
    axes[1].grid(alpha=0.16)
    fig.tight_layout()
    save_figure(fig, "09_cyclotomic_details")


def figure_10_cyclotomic_parseval() -> None:
    z = 2.75 + 0.45j
    a = 0.4
    ell = 6
    children = np.asarray([bernoulli_packet(z, a, ell, child) for child in range(ell)])
    transform = np.fft.fft(children) / math.sqrt(ell)
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.7))
    axes[0].bar(range(ell), np.abs(children) ** 2, color=COLORS * 2, edgecolor=INK)
    axes[0].set_title("child-packet energies")
    axes[0].set_xlabel("child index")
    axes[0].set_ylabel("squared magnitude")
    axes[1].bar(range(ell), np.abs(transform) ** 2, color=COLORS * 2, edgecolor=INK)
    axes[1].set_title("cyclotomic-channel energies")
    axes[1].set_xlabel("channel index")
    total_child = float(np.sum(np.abs(children) ** 2))
    total_transform = float(np.sum(np.abs(transform) ** 2))
    fig.suptitle(f"Parseval across the special-function filter bank: {total_child:.8f} = {total_transform:.8f}",
     fontsize=16.5, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.90])
    save_figure(fig, "10_cyclotomic_parseval")


def figure_11_bernoulli_euler_channels() -> None:
    x = sp.symbols("x")
    grid = np.linspace(-0.2, 1.2, 600)
    n = 7
    low, high = bernoulli_euler_channels(n, x)
    child0 = 2 ** (n - 1) * sp.bernoulli(n, x / 2)
    child1 = 2 ** (n - 1) * sp.bernoulli(n, (x + 1) / 2)
    functions = [sp.lambdify(x, expression, "numpy") for expression in (child0, child1, low, high)]
    values = [function(grid) for function in functions]
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
    axes[0].plot(grid, values[0], color=BLUE, lw=1.8, label="child 0")
    axes[0].plot(grid, values[1], color=ORANGE, lw=1.8, label="child 1")
    axes[0].set_title("two Bernoulli children")
    axes[0].set_xlabel("a")
    axes[0].grid(alpha=0.16)
    axes[0].legend()
    axes[1].plot(grid, values[2], color=TEAL, lw=2, label=r"low: $B_n(a)/\sqrt{2}$")
    axes[1].plot(grid, values[3], color=MAGENTA, lw=2, label=r"high: $-nE_{n-1}(a)/(2\sqrt{2})$")
    axes[1].set_title("unitary low-pass and high-pass channels")
    axes[1].set_xlabel("a")
    axes[1].grid(alpha=0.16)
    axes[1].legend()
    fig.suptitle("Bernoulli and Euler polynomials arise from one exact two-channel refinement", fontsize=16.5,
     weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "11_bernoulli_euler_channels")


def figure_12_packet_wronskians() -> None:
    x = sp.symbols("x")
    grid = np.linspace(-1.2, 2.2, 900)
    fig, axes = plt.subplots(2, 2, figsize=(13.0, 8.0), sharex=True)
    for ax, n, color in zip(axes.ravel(), (3, 4, 5, 6), COLORS[:4], strict=True):
        polynomial = packet_wronskian_polynomial(n, x)
        function = sp.lambdify(x, polynomial, "numpy")
        values = np.asarray(function(grid), dtype=np.float64)
        ax.axhline(0, color=GRID, lw=1)
        ax.plot(grid, values, color=color, lw=1.8)
        roots = [complex(root) for root in sp.nroots(polynomial, maxsteps=200)]
        real_roots = sorted(root.real for root in roots if abs(root.imag) < 1e-9 and -1.2 <= root.real <= 2.2)
        if real_roots:
```

```
            ax.scatter(real_roots, np.zeros(len(real_roots)), color=INK, s=24, zorder=4)
        ax.set_title(rf"$\mathcal{{W}}_{{{n}}}(a)$, {len(real_roots)} displayed real roots")
        ax.grid(alpha=0.14)
    for ax in axes[-1]:
        ax.set_xlabel("a")
    fig.suptitle("Packet Wronskians provide a direct zero and interlacing diagnostic", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.94])
    save_figure(fig, "12_packet_wronskians")


def figure_13_wronskian_root_counts() -> None:
    x = sp.symbols("x")
    degrees = []
    real_counts = []
    nonreal_counts = []
    ns = list(range(2, 17))
    for n in ns:
        polynomial = packet_wronskian_polynomial(n, x)
        roots = [complex(root) for root in sp.nroots(polynomial, maxsteps=400)]
        real_count = sum(abs(root.imag) < 1e-8 for root in roots)
        degree = sp.degree(polynomial, x)
        degrees.append(degree)
        real_counts.append(real_count)
        nonreal_counts.append(degree - real_count)
    fig, ax = plt.subplots(figsize=(12.8, 4.8))
    ax.bar(ns, real_counts, color=TEAL, label="real roots")
    ax.bar(ns, nonreal_counts, bottom=real_counts, color=ORANGE, label="nonreal roots")
    ax.set_xlabel("order n")
    ax.set_ylabel("root count")
    ax.set_title("A useful negative control: packet Wronskians are not universally real-rooted")
    ax.legend()
    ax.grid(axis="y", alpha=0.18)
    fig.tight_layout()
    save_figure(fig, "13_wronskian_root_counts")


def figure_14_cyclotomic_norms() -> None:
    rows = write_cyclotomic_norm_table()
    ns = np.asarray([int(row["n"]) for row in rows])
    norms = np.asarray([float(sp.Rational(row["norm"])) for row in rows])
    fig, ax = plt.subplots(figsize=(12.8, 4.8))
    ax.semilogy(ns, norms, marker="o", color=BLUE, lw=1.8)
    ax.set_xlabel("integer order n")
    ax.set_ylabel("cyclotomic norm")
    ax.set_title("Exact norm of the first non-DC L=3 Bernoulli detail")
    ax.grid(alpha=0.18)
    fig.tight_layout()
    save_figure(fig, "14_cyclotomic_norms")


def figure_15_mobius_kernel_localization(mu: np.ndarray) -> None:
    n = np.arange(2, 6000, dtype=np.float64)
    fig, ax = plt.subplots(figsize=(12.8, 5.0))
    for k, color in zip((100, 500, 2000, 8000), COLORS[:4], strict=True):
        weight = n ** -2 * np.exp(k * np.log1p(-1.0 / (n * n)))
        normalized = weight / weight.max()
        ax.plot(n / math.sqrt(k), normalized, lw=1.8, color=color, label=f"k={k}")
    ax.set_xlim(0, 5)
    ax.set_xlabel(r"scaled integer address $n/\sqrt{k}$")
    ax.set_ylabel("normalized packet weight")
    ax.set_title(r"The Bernoulli-Mobius transfer localizes at the scale $n\asymp\sqrt{k}$")
    ax.grid(alpha=0.18)
    ax.legend()
    fig.tight_layout()
    save_figure(fig, "15_mobius_kernel_localization")


def figure_16_mertens_wavelet() -> None:
    x = np.linspace(1.001, 180, 2000)
    fig, ax = plt.subplots(figsize=(12.8, 5.0))
    for k, color in zip((100, 500, 2000), (BLUE, TEAL, MAGENTA), strict=True):
```

```
        psi = 2 * x ** -5 * np.power(1 - x ** -2, k - 1) * (x * x - k - 1)
        scale = np.max(np.abs(psi))
        ax.plot(x / math.sqrt(k + 1), psi / scale, lw=1.8, color=color, label=f"k={k}")
    ax.axvline(1, color=INK, ls="--", lw=1.2, label=r"sign change $x=\sqrt{k+1}$")
    ax.axhline(0, color=GRID, lw=1)
    ax.set_xlim(0, 3.5)
    ax.set_xlabel(r"scaled coordinate $x/\sqrt{k+1}$")
    ax.set_ylabel("normalized wavelet")
    ax.set_title("The exact finite-difference kernel is a one-sign-change Mertens wavelet")
    ax.legend()
    ax.grid(alpha=0.16)
    fig.tight_layout()
    save_figure(fig, "16_mertens_wavelet")


def figure_17_energy_asymptotic(data: dict[str, np.ndarray]) -> None:
    k = data["k"].astype(float)
    constant = 3 / (2 ** 2.5 * math.pi ** 1.5)
    fig, ax = plt.subplots(figsize=(12.8, 4.9))
    ax.semilogx(k, data["energy"] * k ** 1.5, color=BLUE, lw=2, marker="o", markersize=3, label=r"$k^{3/2}E_k$")
    ax.axhline(constant, color=RED, ls="--", lw=1.8, label=f"limit {constant:.10f}")
    ax.set_xlabel("k")
    ax.set_ylabel("scaled energy")
    ax.set_title("The diagonal Mobius packet energy already has the RH exponent")
    ax.grid(alpha=0.18)
    ax.legend()
    fig.tight_layout()
    save_figure(fig, "17_energy_asymptotic")


def figure_18_mass_and_dimension(data: dict[str, np.ndarray]) -> None:
    k = data["k"].astype(float)
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
    axes[0].loglog(k, data["mass"], color=TEAL, lw=2, marker="o", markersize=3, label="squarefree packet mass")
    axes[0].loglog(k, 3 / math.pi ** 1.5 * k ** -0.5, color=INK, ls="--", label=r"$3\pi^{-3/2}k^{-1/2}$")
    axes[0].set_xlabel("k")
    axes[0].set_ylabel("mass")
    axes[0].set_title("active squarefree mass")
    axes[0].grid(alpha=0.18, which="both")
    axes[0].legend()
    axes[1].loglog(k, data["effective_dimension"], color=MAGENTA, lw=2, marker="o", markersize=3)
    axes[1].set_xlabel("k")
    axes[1].set_ylabel(r"$S_k^2/E_k$")
    axes[1].set_title("effective number of active packets")
    axes[1].grid(alpha=0.18, which="both")
    fig.suptitle(r"The packet cloud grows like a constant multiple of $\sqrt{k}$", fontsize=16.5, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "18_mass_and_dimension")


def figure_19_coherence_ratio(data: dict[str, np.ndarray]) -> None:
    k = data["k"].astype(float)
    fig, axes = plt.subplots(1, 2, figsize=(13.0, 4.8))
    axes[0].loglog(k, np.abs(data["c"]), color=BLUE, lw=1.8, marker="o", markersize=3, label=r"$|c_k|$")
    axes[0].loglog(k, np.sqrt(data["energy"]), color=TEAL, lw=1.8, label=r"$\sqrt{E_k}$")
    axes[0].set_xlabel("k")
    axes[0].set_ylabel("amplitude")
    axes[0].set_title("coherent amplitude and diagonal scale")
    axes[0].grid(alpha=0.18, which="both")
    axes[0].legend()
    axes[1].semilogx(k, data["coherence"], color=MAGENTA, lw=1.8, marker="o", markersize=3)
    axes[1].set_xlabel("k")
    axes[1].set_ylabel(r"$\Gamma_k=|c_k|^2/E_k$")
    axes[1].set_title("Mobius packet coherence")
    axes[1].grid(alpha=0.18)
    fig.suptitle("RH is equivalent to subpolynomial growth of the coherence ratio", fontsize=16.5, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "19_coherence_ratio")


def figure_20_log_time_signal(data: dict[str, np.ndarray]) -> None:
```

```
    k = data["k"].astype(float)
    y = k ** 0.75 * data["c"]
    fig, ax = plt.subplots(figsize=(12.8, 4.8))
    ax.plot(np.log(k), y, color=BLUE, lw=1.4, marker="o", markersize=3)
    ax.axhline(0, color=GRID, lw=1)
    ax.set_xlabel(r"logarithmic time $u=\log k$")
    ax.set_ylabel(r"$k^{3/4}c_k$")
    ax.set_title("Normalized Bernoulli-difference signal in logarithmic time")
    ax.grid(alpha=0.18)
    fig.tight_layout()
    save_figure(fig, "20_log_time_signal")


def figure_21_long_time_power(mu: np.ndarray) -> None:
    nmax = 60000
    n = np.arange(2, nmax + 1, dtype=np.float64)
    mu_n = mu[2 : nmax + 1].astype(np.float64)
    k = 1200
    coeff = mu_n / (n * n) * np.exp(k * np.log1p(-1.0 / (n * n)))
    t_values = np.linspace(0, 160, 500)
    signal = np.zeros_like(t_values, dtype=np.complex128)
    chunk = 5000
    logs = np.log(n)
    for start in range(0, len(n), chunk):
        stop = min(start + chunk, len(n))
        phase = np.exp(-1j * np.outer(t_values, logs[start:stop]))
        signal += phase @ coeff[start:stop]
    energy = float(np.sum(coeff * coeff))
    running_power = np.cumsum(np.abs(signal) ** 2) / np.arange(1, len(signal) + 1)
    fig, axes = plt.subplots(2, 1, figsize=(12.8, 7.0), sharex=True)
    axes[0].plot(t_values, signal.real, color=BLUE, lw=1.2, label="real")
    axes[0].plot(t_values, signal.imag, color=ORANGE, lw=1.2, label="imaginary")
    axes[0].set_ylabel("F_k(t)")
    axes[0].set_title("Mobius-Bernoulli quantum correlation")
    axes[0].legend()
    axes[0].grid(alpha=0.15)
    axes[1].plot(t_values, running_power, color=TEAL, lw=1.7, label="running mean power")
    axes[1].axhline(energy, color=RED, ls="--", label=f"diagonal energy {energy:.3e}")
    axes[1].set_xlabel("time t")
    axes[1].set_ylabel("mean squared magnitude")
    axes[1].grid(alpha=0.15)
    axes[1].legend()
    fig.tight_layout()
    save_figure(fig, "21_long_time_power")


def figure_22_polyphase_mobius_packets(mu: np.ndarray) -> None:
    modulus = 12
    ks = [200, 1000, 5000]
    fig, axes = plt.subplots(len(ks), 1, figsize=(12.8, 8.3), sharex=True)
    for ax, k in zip(axes, ks, strict=True):
        packets = polyphase_transfer_packets(k, modulus, 180000, mu)
        colors = [TEAL if value >= 0 else ORANGE for value in packets]
        ax.bar(range(modulus), packets, color=colors, edgecolor=INK)
        ax.axhline(0, color=GRID, lw=1)
        ax.set_ylabel(f"k={k}")
        ax.grid(axis="y", alpha=0.15)
    axes[-1].set_xlabel("integer residue modulo 12")
    fig.suptitle("Exact arithmetic polyphase packets of the Mobius transfer signal", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.94])
    save_figure(fig, "22_polyphase_mobius_packets")


def figure_23_character_packets(mu: np.ndarray) -> None:
    nmax = 160000
    character = quadratic_character_mod5(nmax)
    k_values = np.unique(np.logspace(1, 4.6, 80).astype(int))
    principal = []
    quadratic = []
    for k in k_values:
        principal.append(generalized_transfer_direct(2, 2, int(k), 0, nmax, mu))
```

```
        quadratic.append(generalized_transfer_direct(2, 2, int(k), 0, nmax, mu, character))
    fig, ax = plt.subplots(figsize=(12.8, 4.9))
    ax.loglog(k_values, np.abs(principal), color=BLUE, lw=1.8, label="principal character")
    ax.loglog(k_values, np.abs(quadratic), color=MAGENTA, lw=1.8, label="quadratic character mod 5")
    ax.set_xlabel("k")
    ax.set_ylabel("transfer amplitude")
    ax.set_title("Character packets create a natural GRH-facing atlas")
    ax.grid(alpha=0.18, which="both")
    ax.legend()
    fig.tight_layout()
    save_figure(fig, "23_character_packets")


def figure_24_quantum_operator_pipeline() -> None:
    fig, ax = plt.subplots(figsize=(12.8, 6.1))
    ax.axis("off")
    stages = [
        (0.03, r"$|\phi_a\rangle$", "power-law state", BLUE),
        (0.23, r"$e^{-kK_\beta}$", "scale filter", TEAL),
        (0.43, r"$\mathcal{M}_\chi$", "Mobius-character sign", ORANGE),
        (0.63, r"$e^{-itH}$", "logarithmic evolution", MAGENTA),
        (0.83, r"$\langle\phi_a|$", "coherence readout", RED),
    ]
    for x0, formula, title, color in stages:
        box = FancyBboxPatch((x0, 0.40), 0.14, 0.22, boxstyle="round,pad=.015", facecolor=color + "24", edgecolor=
     color, lw=2)
        ax.add_patch(box)
        ax.text(x0 + 0.07, 0.55, formula, ha="center", va="center", fontsize=15)
        ax.text(x0 + 0.07, 0.45, title, ha="center", va="center", fontsize=9.5)
    for left, right in zip(stages[:-1], stages[1:], strict=True):
        ax.add_patch(FancyArrowPatch((left[0] + 0.14, 0.51), (right[0], 0.51), arrowstyle="-|>", mutation_scale=14,
     lw=1.8, color=INK))
    ax.text(0.5, 0.82, r"$\mathcal{C}_{a,\beta,\chi}(k,t)=\langle\phi_a|\mathcal{M}_\chi e^{-kK_\beta}e^{-itH}|\
     phi_a\rangle$", ha="center", fontsize=18, weight="bold")
    ax.text(0.5, 0.18, r"$H|n\rangle=\log n|n\rangle$ and $K_\beta=-\log(I-e^{-\beta H})$", ha="center", fontsize=14,
      color=MUTED)
    save_figure(fig, "24_quantum_operator_pipeline")


def figure_25_function_family_map() -> None:
    fig, ax = plt.subplots(figsize=(13.0, 7.0))
    ax.axis("off")
    nodes = {
        "scalar": (0.10, 0.74, "scalar analytic\nfunction", BLUE),
        "lift": (0.38, 0.74, "polyphase\nvector lift", TEAL),
        "matrix": (0.68, 0.74, "matrix differential\nsystem", ORANGE),
        "bern": (0.12, 0.36, "Bernoulli-Hurwitz\npackets", MAGENTA),
        "lerch": (0.40, 0.36, "cyclotomic Lerch\ndetails", RED),
        "wronskian": (0.68, 0.36, "packet Wronskians\nand zero tests", YELLOW),
        "mobius": (0.88, 0.36, "Mobius-Bernoulli\ncoherence", BLUE),
    }
    for key, (x0, y0, label, color) in nodes.items():
        ax.add_patch(FancyBboxPatch((x0 - 0.09, y0 - 0.06), 0.18, 0.12, boxstyle="round,pad=.012", facecolor=color +
      "22", edgecolor=color, lw=2))
        ax.text(x0, y0, label, ha="center", va="center", fontsize=11, weight="bold")
    edges = [
        ("scalar", "lift"), ("lift", "matrix"), ("scalar", "bern"), ("bern", "lerch"), ("lerch", "wronskian"), ("
     wronskian", "mobius"), ("matrix", "mobius"),
    ]
    for source, target in edges:
        sx, sy = nodes[source][0], nodes[source][1]
        tx, ty = nodes[target][0], nodes[target][1]
        ax.add_patch(FancyArrowPatch((sx, sy), (tx, ty), arrowstyle="-|>", mutation_scale=12, lw=1.4, color=GRID,
     connectionstyle="arc3,rad=0.05"))
    ax.text(0.5, 0.93, "A special-function factory generated by exact packet analysis", ha="center", fontsize=20,
     weight="bold")
    ax.text(0.5, 0.10, "The constructions share reconstruction, refinement, transfer, and certificate laws", ha="
     center", fontsize=13, color=MUTED)
    save_figure(fig, "25_function_family_map")


```

```
def figure_26_status_map() -> None:
    fig, ax = plt.subplots(figsize=(12.8, 5.9))
    ax.axis("off")
    categories = [
        (0.08, 0.65, 0.24, 0.20, "PROVED HERE", ["polyphase lift", "Bernoulli packet refinement", "cyclotomic inverse transform", "energy asymptotic"], TEAL),
        (0.38, 0.65, 0.24, 0.20, "CLASSICAL INPUT", ["Hurwitz zeta", "Raabe multiplication", "Lerch transcendent", "Baez-Duarte criterion"], BLUE),
        (0.68, 0.65, 0.24, 0.20, "COMPUTATIONAL", ["Wronskian root maps", "cyclotomic norms", "coherence plots", "character packets"], ORANGE),
        (0.23, 0.24, 0.24, 0.20, "OPEN", ["Wronskian sign laws", "zero interlacing", "p-adic detail bounds", "character-uniform coherence"], MAGENTA),
        (0.53, 0.24, 0.24, 0.20, "MAJOR CONJECTURE", ["subpolynomial Mobius coherence", "RH and GRH applications", "positive canonical systems"], RED),
    ]
    for x0, y0, w, h, title, entries, color in categories:
        ax.add_patch(FancyBboxPatch((x0, y0), w, h, boxstyle="round,pad=.015", facecolor=color + "20", edgecolor=color, lw=2))
        ax.text(x0 + w / 2, y0 + h - 0.035, title, ha="center", va="center", fontsize=11.5, weight="bold", color=color)
        for index, entry in enumerate(entries):
            ax.text(x0 + 0.025, y0 + h - 0.075 - 0.030 * index, "* " + entry, ha="left", va="center", fontsize=9.4)
    ax.text(0.5, 0.94, "Claim boundary for the generated function theory", ha="center", fontsize=19, weight="bold")
    save_figure(fig, "26_status_map")


def generate_all_figures() -> dict[str, object]:
    checks = run_symbolic_checks()
    if not all(check.passed for check in checks):
        failed = [check for check in checks if not check.passed]
        raise RuntimeError(f"symbolic checks failed: {failed}")

    nmax = 200000
    mu = mobius_sieve(nmax)
    k_values = np.unique(np.logspace(1, 5, 92).astype(int))
    transfer_data = principal_transfer_values(k_values, nmax, mu)

    figure_01_polyphase_partition()
    figure_02_carry_shift()
    figure_03_exp_sections()
    figure_04_exp_reconstruction()
    figure_05_operator_lift_map()
    figure_06_bessel_sections()
    figure_07_bernoulli_packet_reconstruction()
    figure_08_bernoulli_refinement_tree()
    figure_09_cyclotomic_details()
    figure_10_cyclotomic_parseval()
    figure_11_bernoulli_euler_channels()
    figure_12_packet_wronskians()
    figure_13_wronskian_root_counts()
    figure_14_cyclotomic_norms()
    figure_15_mobius_kernel_localization(mu)
    figure_16_mertens_wavelet()
    figure_17_energy_asymptotic(transfer_data)
    figure_18_mass_and_dimension(transfer_data)
    figure_19_coherence_ratio(transfer_data)
    figure_20_log_time_signal(transfer_data)
    figure_21_long_time_power(mu)
    figure_22_polyphase_mobius_packets(mu)
    figure_23_character_packets(mu)
    figure_24_quantum_operator_pipeline()
    figure_25_function_family_map()
    figure_26_status_map()

    # Numerical result ledger
    constant = 3 / (2 ** 2.5 * math.pi ** 1.5)
    large_index = -1
    energy_scaled = float(transfer_data["energy"][large_index] * transfer_data["k"][large_index] ** 1.5)
    mass_scaled = float(transfer_data["mass"][large_index] * math.sqrt(transfer_data["k"][large_index]))

    quantum_k = 1200
```

```
    mean_power = long_time_power(2, 2, quantum_k, nmax, mu)
    zero_time = generalized_transfer_direct(2, 2, quantum_k, 0.0, nmax, mu)

    results = {
        "figure_pairs": 26,
        "symbolic_checks": len(checks),
        "symbolic_checks_passed": sum(check.passed for check in checks),
        "mobius_nmax": nmax,
        "largest_k": int(transfer_data["k"][large_index]),
        "energy_scaled_at_largest_k": energy_scaled,
        "energy_asymptotic_constant": constant,
        "energy_relative_error": abs(energy_scaled - constant) / constant,
        "mass_scaled_at_largest_k": mass_scaled,
        "mass_asymptotic_constant": 3 / math.pi ** 1.5,
        "quantum_k": quantum_k,
        "quantum_zero_time_abs": abs(zero_time),
        "quantum_long_time_diagonal_power": mean_power,
        "quantum_coherence_ratio": abs(zero_time) ** 2 / mean_power,
    }

    with (RESULTS / "results.json").open("w", encoding="utf-8") as handle:
        json.dump(results, handle, indent=2, sort_keys=True)

    with (RESULTS / "checks.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=["name", "passed", "detail"])
        writer.writeheader()
        writer.writerows([{"name": check.name, "passed": check.passed, "detail": check.detail} for check in checks])

    with (TABLES / "mobius_transfer_samples.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.writer(handle)
        writer.writerow(["k", "c_k", "energy", "mass", "coherence", "effective_dimension"])
        for row in zip(
            transfer_data["k"],
            transfer_data["c"],
            transfer_data["energy"],
            transfer_data["mass"],
            transfer_data["coherence"],
            transfer_data["effective_dimension"],
            strict=True,
        ):
            writer.writerow(row)

    return results


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def main() -> None:
    parser = argparse.ArgumentParser(description="Generate the Projector-Generated Special Functions paper assets")
    parser.add_argument("--base", type=Path, default=ROOT)
    args = parser.parse_args()
    if args.base.resolve() != ROOT.resolve():
        raise SystemExit("This release expects to run from its source directory")
    results = generate_all_figures()
    print(json.dumps(results, indent=2, sort_keys=True))


if __name__ == "__main__":
    main()
```

Chapter C

Bibliographic note

Bibliography

[1] NIST Digital Library of Mathematical Functions, Chapter 24, “Bernoulli and Euler Polynomials,” particularly Section 24.4 on difference, derivative, and multiplication formulas.

[2] NIST Digital Library of Mathematical Functions, Sections 25.11 and 25.14 on the Hurwitz zeta function and Lerch transcendent.

[3] L. Báez-Duarte, “A Sequential Riesz-like Criterion for the Riemann Hypothesis,” *International Journal of Mathematics and Mathematical Sciences*, 2005, Article 141565, 3527–3537.

[4] J. Cisło and M. Wolf, “On the Riesz and Báez-Duarte Criteria for the Riemann Hypothesis,” 2008, arXiv:0807.2971.

[5] M. Garg and B. Maji, “Hardy-Littlewood-Riesz Type Equivalent Criteria for the Generalized Riemann Hypothesis,” 2022, arXiv:2208.07596.

[6] S. Gupta and A. Vatwani, “Riesz Type Criteria for L-Functions in the Selberg Class,” 2022, arXiv:2211.02954.

[7] S. Wei, Y. Zhai, Q. Lu, W. Yang, P. Gao, C. Wei, J. Song, F. Nori, T. Xin, and G. Long, “The Riemann Hypothesis Manifested in Dynamical Quantum Phase Transitions,” *Nature Communications*, accepted 18 June 2026, DOI: 10.1038/s41467-026-74935-8.

[8] Komorebi P., “Bernoulli Transfer-Block Reformulation of Zeta,” unpublished proof-development note.